



Staqtapp-TDS

Programmer Core API Guide

Direct implementation calls for .tds storage, variables, trace ranking, CSV operations, Driver systems, Browser, and Driver Studio

Release	v3.5.2
Audience	AI-system programmers and integration engineers
Primary contract	Result-first operations with explicit authority boundaries
Source basis	Public v3.5.2 modules and validated usage tests

Use this guide first. The separate API Surface Reference remains the exhaustive class inventory.

How to use this guide

This document is organized by programming task rather than by source-file order. Start with the quick recipes, then use the call maps for exact signatures. The API index includes public operational functions and the methods that perform storage, validation, execution, review, export, or observation. Passive dataclasses are introduced where programmers construct them, but their repetitive value-only properties are not expanded into noise.

Three call styles

Result-first: use methods such as `write_result()`, `read_result()`, and `rank_trace_result()` in long-running AI services.

Direct-value: use `read_value()`, `read_text()`, and typed loaders when missing/corrupt data should be exceptional.

Prepare / commit / replay: CSV and semantic contracts make mutation authority explicit. Prepare is generally read-only; commit persists; replay detects drift.

All signatures in the reference sections are extracted from the v3.5.2 source tree used to build this PDF.

Contents and PDF bookmarks

Use the document outline/bookmarks for immediate navigation. The major task-oriented sections are:

1. Ten-minute implementation path
2. Result-first programming model
3. Directory storage and entry operations
4. .tds persistence, mmap readers, and mounting
5. Variables, text, JSON, and serialization
6. Provenance, verification, telemetry, diagnostics, and recovery
7. Trace storage, provenance, and ranking
8. CSV Suite: direct operational workflow
9. CSV import, validation, export, and transactions
10. CSV scanning, row anchors, and artifact security
11. CSV storage bridge, native evidence, and performance gates
12. CSV Interpole and centralized Browser evidence
13. CSV Semantic IR candidates and lifecycle review
14. CSV operational call index
15. Driver TDDL, bytecode, Foundry, and Runtime Manager
16. Driver regression, review, evidence, and performance
17. Driver Studio and PyQt5 integration
18. Driver and Studio operational function index
19. Centralized Browser and local admin control
20. Configuration, native management, security, and low-level calls
21. Integration recipes
22. API selection matrix
23. Stable boundaries to preserve
24. Validation basis and compatibility
25. Final programmer checklist

1. Ten-minute implementation path

1.1 Store AI state and flush to .tds

```
from pathlib import Path
from staqtapp_tds import TDSFileSystem, TDSPersistence

fs = TDSFileSystem("agent_state")
runtime = fs.makedirs("/models/runtime")

runtime.write_text("system_prompt", "You are a careful planning agent.")
runtime.write_json("settings", {"temperature": 0.2, "tools": True})
runtime.write_result("step_count", 7)

result = runtime.read_result("settings")
if not result.ok:
    raise RuntimeError(f"{result.code}: {result.message}")

store = TDSPersistence(Path("./tds_store"))
store.flush(fs, parallel_nodes=False)

loaded = store.load_node(
    Path("./tds_store/agent_state__models__runtime.tds")
)
assert loaded.read_value("step_count") == 7
```

Persistence naming

Each TDS directory node is persisted as its own .tds file. Nested directory names are joined with double underscores in the default persistence layout. Use the mapping returned by `TDSPersistence.flush()` or inspect the mount directory rather than manually guessing names in production code.

1.2 Controlled variables and stalk chains

```
state = fs.makedirs("/agent/state")

state.addvar("reward", 1.0)
state.editvar("reward", 1.25)
state.lockvar("reward")

found = state.findvar("reward")
assert found.ok and found.value == 1.25

state.unlockvar("reward")
state.addvar("context", ["initial"])
state.stalkvar("~context", ["observation-1"])
state.stalkvar("~context", ["observation-2"])

latest = state.loadvar("context_0002")
state.stalkvar("context") # clear tracked increments; keep the base
```

1.3 Rank trace candidates

```
from staqtapp_tds.spiral import rank_traces

ranked = rank_traces(
    ["trace-a", "trace-b", "trace-c"],
    [0.82, 0.95, 0.95],
    confidences=[0.90, 0.92, 0.92],
    depths=[2, 3, 1],
    limit=2,
)

for item in ranked:
    print(item.rank, item.trace_id, item.rank_score)
```

1.4 Import and validate CSV evidence

```
from staqtapp_tds.csv_layer import (
    export_original_csv,
    import_csv_bytes,
    prove_original_roundtrip,
    validate_csv_artifacts,
)

csv_dir = fs.makedirs("/datasets")
manifest = import_csv_bytes(
    csv_dir,
    b"id,name,score\n1,Ada,99\n2,Grace,98\n",
    source_name="people.csv",
)

report = validate_csv_artifacts(csv_dir, manifest.csv_id)
assert report.ok
assert prove_original_roundtrip(csv_dir, manifest.csv_id).byte_equivalent
original = export_original_csv(csv_dir, manifest.csv_id)
```

1.5 Start the centralized Browser

```
staqtapp-tds-admin status
staqtapp-tds-admin verify --sample
staqtapp-tds-admin serve-panel --host 127.0.0.1 --port 8765
```

Open <http://127.0.0.1:8765/>. The Browser reads snapshots and copied diagnostic events. It is not a polling loop over native index locks.

2. Result-first programming model

TDS distinguishes expected operational failures from programmer errors. Result-first calls return a `TDSResult` containing `ok`, `code`, `message`, `name`, `path`, `value`, and `meta`.

```
result = directory.read_result("agent_state")

if result.ok:
    value = result.value
else:
    logger.warning("TDS operation failed", extra={
        "code": str(result.code),
        "message": result.message,
        "path": result.path,
        "meta": result.meta,
    })
```

Need	Preferred call	Behavior
Long-running service write	<code>TDSDirectory.write_result(...)</code>	Structured success/failure; no application-level exception required for normal conflicts.
Long-running service read	<code>TDSDirectory.read_result(...)</code>	Missing/corrupt data is represented in <code>TDSResult</code> .
Strict internal invariant	<code>TDSDirectory.read_value(...)</code>	Returns the value directly; missing entry raises.
Human-readable text	<code>TDSDirectory.read_text(...)</code>	Returns <code>str</code> and validates the stored payload kind.
Trace ranking evidence	<code>rank_trace_result(...)</code>	Returns ranked records inside <code>TDSResult</code> .
Native engine inspection	<code>native_status_result()</code>	Availability/state is returned without halting the host service.

known_result_codes

```
known_result_codes() -> 'tuple[str, ...]'
```

Import: `staqtapp_tds.result`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

is_known_result_code

```
is_known_result_code(code: 'TDSResultCode | str') -> 'bool'
```

Import: `staqtapp_tds.result`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

result_info

```
result_info(code: 'TDSResultCode | str') -> 'TDSResultInfo | None'
```

Import: `staqtapp_tds.result`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

3. Directory storage and entry operations

The primary integration surface is `TDSFileSystem` plus the `TDSDirectory` returned by `fs.root`, `makedirs()`, `resolve()`, or `mkdir()`.

`TDSFileSystem`

```
TDSFileSystem(name: 'str' = 'tds_root', manifest_policy: 'Optional[ManifestPolicy]' = None, runtime_config: 'Optional[Ru
```

Owns the directory tree, shared configuration, telemetry manager, and observation snapshot surface.

`TDSFileSystem.resolve`

```
resolve(self, path: 'str') -> 'TDSDirectory'
```

Resolve a directory path or raise when it does not exist.

`TDSFileSystem.resolve_radix`

```
resolve_radix(self, path: 'str') -> 'TDSDirectory'
```

Public operational method on this integration surface.

`TDSFileSystem.makedirs`

```
makedirs(self, path: 'str', **kwargs) -> 'TDSDirectory'
```

Create an absolute or relative directory path and return the final `TDSDirectory`.

`TDSFileSystem.parallel_batch_write`

```
parallel_batch_write(self, writes: 'List[Tuple[str, str, Any]]') -> 'None'
```

Write multiple (path, name, value) items using the shared concurrency pool.

`TDSFileSystem.snapshot_headers`

```
snapshot_headers(self) -> 'Dict[str, dict]'
```

Public operational method on this integration surface.

`TDSFileSystem.telemetry_snapshot`

```
telemetry_snapshot(self) -> 'Dict[str, dict]'
```

Public operational method on this integration surface.

`TDSFileSystem.observation_snapshot`

```
observation_snapshot(self, *, force: 'bool' = False) -> 'Dict[str, object]'
```

Public operational method on this integration surface.

`TDSFileSystem.capability_snapshot`

```
capability_snapshot(self) -> 'Dict[str, List[str]]'
```

Public operational method on this integration surface.

`TDSDirectory`

```
TDSDirectory(name: 'str', fmt_id: 'FmtID' = , flags: 'int' = , parent: "Optional['TDSDirectory']" = None, manifest_polic
```

Represents one directory node. Its write/read methods are the normal application API; low-level header/index helpers are advanced integration calls.

`TDSDirectory.write`

```
write(self, name: 'str', value: 'Any', fmt_id: 'FmtID' = , compress: 'bool | None' = False, codec: 'str' = '', provenance: 'Prove
```

Store an entry in the directory.

`TDSDirectory.write_result`

```
write_result(self, name: 'str', value: 'Any', fmt_id: 'FmtID' = , compress: 'bool | None' = False, codec: 'str' = '', pr
```

Store a value and return `TDSResult` instead of requiring exception-driven application control flow.

`TDSDirectory.write_variable`

```
write_variable(self, name: 'str', value: 'Any', *, compress: 'bool | None' = None, codec: 'str' = '', provenance: 'Prove
```

Store the requested specialized payload and return the documented entry/result type.

`TDSDirectory.write_json`

```
write_json(self, name: 'str', value: 'Any', *, overwrite: 'bool' = False, compress: 'bool | None' = None, codec: 'str' = ''
```

Store the requested specialized payload and return the documented entry/result type.

TDSDirectory.write_text

```
write_text(self, name: 'str', text: 'str', *, overwrite: 'bool' = False, compress: 'bool | None' = None, codec: 'str' =
```

Store the requested specialized payload and return the documented entry/result type.

TDSDirectory.write_text_chunked

```
write_text_chunked(self, name: 'str', text: 'str', *, chunk_size: 'int' = 65536, overwrite: 'bool' = False, compress: 'b
```

Store large UTF-8 text as bounded chunks without cutting code points.

TDSDirectory.read

```
read(self, name: 'str') -> 'TDSResult'
```

Read an entry.

TDSDirectory.read_result

```
read_result(self, name: 'str') -> 'TDSResult'
```

Return a TDSResult containing the value or a stable failure code.

TDSDirectory.read_value

```
read_value(self, name: 'str') -> 'Any'
```

Return the stored value directly; use when missing data should be exceptional.

TDSDirectory.read_text

```
read_text(self, name: 'str') -> 'str'
```

Read the requested specialized payload or metadata.

TDSDirectory.read_text_result

```
read_text_result(self, name: 'str') -> 'TDSResult'
```

Read the requested specialized payload or metadata.

TDSDirectory.delete

```
delete(self, name: 'str') -> 'TDSResult'
```

Delete an entry and return result evidence.

TDSDirectory.delete_result

```
delete_result(self, name: 'str') -> 'TDSResult'
```

Public operational method on this integration surface.

TDSDirectory.delete_entry

```
delete_entry(self, name: 'str') -> 'None'
```

Public operational method on this integration surface.

TDSDirectory.mkdir

```
mkdir(self, name: 'str', **kwargs) -> '"TDSDirectory"'
```

Create a child directory.

TDSDirectory.cd

```
cd(self, name: 'str') -> '"TDSDirectory"'
```

Public operational method on this integration surface.

TDSDirectory.ls

```
ls(self, sort_by_prob: 'bool' = True) -> 'List[str]'
```

List entries/children.

TDSDirectory.parallel_read_all

```
parallel_read_all(self) -> 'Dict[str, Any]'
```

Public operational method on this integration surface.

TDSDirectory.recursive_join

```
recursive_join(self, dtype=, max_depth: 'int' = 8) -> 'np.ndarray'
```

Public operational method on this integration surface.

TDSDirectory.build_offset_index

```
build_offset_index(self) -> 'np.ndarray'
```

Public operational method on this integration surface.

TDSDirectory.entry_metadata

```
entry_metadata(self, name: 'str') -> 'dict'
```

Public operational method on this integration surface.

TDSDirectory.entry_metadata_result

```
entry_metadata_result(self, name: 'str') -> 'TDSResult'
```

Public operational method on this integration surface.

TDSDirectory.invariant_report

```
invariant_report(self) -> 'dict'
```

Public operational method on this integration surface.

TDSDirectory.provenance_record

```
provenance_record(self, name: 'str') -> 'np.ndarray'
```

Public operational method on this integration surface.

TDSDirectory.provenance_record_result

```
provenance_record_result(self, name: 'str') -> 'TDSResult'
```

Public operational method on this integration surface.

TDSDirectory.header_bytes

```
header_bytes(self) -> 'bytes'
```

Public operational method on this integration surface.

TDSDirectory.to_bytes

```
to_bytes(self) -> 'bytes'
```

Public operational method on this integration surface.

TDSDirectory.path

```
path(self) -> 'str'
```

Public operational method on this integration surface.

TDSDirectory.telemetry_snapshot

```
telemetry_snapshot(self) -> 'dict'
```

Public operational method on this integration surface.

TDSDirectory.capability_names

```
capability_names(self) -> 'List[str]'
```

Public operational method on this integration surface.

TDSDirectory.is_reserved_namespace

```
is_reserved_namespace(self, name: 'str') -> 'bool'
```

Public operational method on this integration surface.

TDSDirectory.reserved_namespace_names

```
reserved_namespace_names(self) -> 'List[str]'
```

Public operational method on this integration surface.

TDSDirectory.srz_record

```
srz_record(self) -> 'np.ndarray'
```

Public operational method on this integration surface.

Format selection

Use `write_text()` for text, `write_json()` for JSON-safe values, and `write_result()` or `write()` for general Python values. Restricted pickle compatibility remains behind the Serialization Manager and PicklePolicy boundary.

4. .tds persistence, mmap readers, and mounting

TDS persistence serializes each directory node into a compact .tds file with a fixed header, data block, slot index, and optional integrity sidecar. Writers use temporary files and atomic replacement. Readers validate file geometry, slot bounds, index hashes, metadata epoch, and content hashes before returning values.

TDSPersistence

```
TDSPersistence(mount_dir, *, create_manifest: 'bool' = True)
```

High-level persistence manager for complete TDSFileSystem trees and mounted readers.

TDSPersistence.flush_node

```
flush_node(self, node: 'TDSDirectory', parallel: 'bool' = False) -> 'Tuple[str, int]'
```

Public operational method on this integration surface.

TDSPersistence.flush

```
flush(self, fs: 'TDSFileSystem', parallel_nodes: 'bool' = True) -> 'Dict[str, int]'
```

Flush all filesystem nodes to .tds files using atomic replacement.

TDSPersistence.load_node

```
load_node(self, tds_path, into: 'Optional[TDSDirectory]' = None) -> 'TDSDirectory'
```

Public operational method on this integration surface.

TDSPersistence.mount

```
mount(self, fs: 'TDSFileSystem') -> 'None'
```

Attach persisted readers to an existing filesystem for lazy direct reads.

TDSPersistence.unmount

```
unmount(self) -> 'Dict[str, int]'
```

Public operational method on this integration surface.

TDSPersistence.close_reader

```
close_reader(self, tds_path) -> 'None'
```

Public operational method on this integration surface.

TDSPersistence.open_readers

```
open_readers(self, paths) -> 'List[TDSReader]'
```

Public operational method on this integration surface.

TDSWriter

```
TDSWriter(path)
```

Direct writer for one TDSDirectory and optional recursive descendants.

TDSWriter.write

```
write(self, directory: 'TDSDirectory', recurse: 'bool' = True) -> 'int'
```

Store an entry in the directory.

TDSWriter.write_parallel

```
write_parallel(self, directory: 'TDSDirectory') -> 'int'
```

Store the requested specialized payload and return the documented entry/result type.

TDSReader

```
TDSReader(path)
```

Validated mmap random-access reader for one .tds file.

TDSReader.reload

```
reload(self) -> 'None'
```

Public operational method on this integration surface.

TDSReader.read

```
read(self, name: 'str', *, codec: 'str | None' = None, content_hash: 'str | None' = None) -> 'Any'
```

Read an entry.

TDSReader.read_raw

```
read_raw(self, name: 'str') -> 'bytes'
```

Return the stored raw slot payload bytes.

TDSReader.read_result

```
read_result(self, name: 'str') -> 'TDSResult'
```

Read the requested specialized payload or metadata.

TDSReader.read_many

```
read_many(self, names: 'List[str]') -> 'Dict[str, Any]'
```

Read the requested specialized payload or metadata.

TDSReader.read_many_result

```
read_many_result(self, names: 'List[str]') -> 'TDSResult'
```

Read the requested specialized payload or metadata.

TDSReader.keys

```
keys(self) -> 'List[str]'
```

Public operational method on this integration surface.

TDSReader.close

```
close(self) -> 'None'
```

Release reader resources.

ParallelFlusher

```
ParallelFlusher(mount_dir)
```

Queue multiple directory writes and complete them as a bounded batch.

ParallelFlusher.enqueue

```
enqueue(self, node: 'TDSDirectory') -> 'None'
```

Public operational method on this integration surface.

ParallelFlusher.flush_all

```
flush_all(self, parallel: 'bool' = True) -> 'Dict[str, int]'
```

Complete all queued persistence writes.

```
from staqtapp_tds import TDSReader

reader = TDSReader("../tds_store/agent_state__models__runtime.tds")
try:
    # Tree flushes index entries by their fully qualified slot key.
    print(reader.keys())
    value = reader.read("/agent_state/models/runtime/settings")
    safe = reader.read_result(
        "/agent_state/models/runtime/missing_optional_value"
    )
finally:
    reader.close()
```

Reader slot names

For files produced by a complete tree flush, `TDSReader.keys()` returns fully qualified slot keys such as `/agent_state/models/runtime/settings`. Use those keys for `read()` and `read_many()`.

`TDSPersistence.load_node()` reconstructs a directory and restores convenient short-name reads.

Integrity failures

Header, index, slot-range, sidecar, epoch, codec, and content-hash failures are fail-closed. Do not bypass `TDSReader` by slicing the file directly unless you are implementing a separate forensic tool.

5. Variables, text, JSON, and serialization

5.1 VariableControl

VariableControl

```
VariableControl(directory: 'Any', *, error_mode: 'ErrorLogMode' = )
```

Can be created directly, but the same calls are exposed on TDSDirectory and are normally more convenient.

VariableControl.is_locked

```
is_locked(self, name: 'str') -> 'bool'
```

Public operational method on this integration surface.

VariableControl.addvar

```
addvar(self, name: 'str', data: 'Any') -> 'TDSResult'
```

Public operational method on this integration surface.

VariableControl.editvar

```
editvar(self, name: 'str', data: 'Any', *, overwrite: 'bool' = True) -> 'TDSResult'
```

Public operational method on this integration surface.

VariableControl.lockvar

```
lockvar(self, name: 'str', locked: 'bool' = True) -> 'TDSResult'
```

Public operational method on this integration surface.

VariableControl.unlockvar

```
unlockvar(self, name: 'str') -> 'TDSResult'
```

Public operational method on this integration surface.

VariableControl.stalkvar

```
stalkvar(self, name: 'str', data: 'Any' = None) -> 'TDSResult'
```

Public operational method on this integration surface.

VariableControl.findvar

```
findvar(self, name: 'str') -> 'TDSResult'
```

Public operational method on this integration surface.

VariableControl.loadvar

```
loadvar(self, name: 'str') -> 'Any'
```

Public operational method on this integration surface.

VariableControl.snapshot

```
snapshot(self) -> 'Dict[str, Any]'
```

Return a JSON-safe snapshot.

VariableControl.restore

```
restore(self, snap: 'Dict[str, Any]') -> 'None'
```

Restore controller state from a prior snapshot.

5.2 Payload and serialization helpers

choose_variable_kind

```
choose_variable_kind(value: 'Any') -> 'PayloadKind'
```

Import: staqtapp_tds.serializers

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

content_hash_bytes

```
content_hash_bytes(raw: 'bytes', algorithm: 'str' = 'sha256') -> 'str'
```

Import: staqtapp_tds.serializers

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

dumps_pickle

```
dumps_pickle(value: 'Any', policy: 'PicklePolicy | None' = None) -> 'bytes'
```

Import: staqtapp_tds.tds_pickle

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

loads_pickle

```
loads_pickle(raw: 'bytes', policy: 'PicklePolicy | None' = None) -> 'Any'
```

Import: staqtapp_tds.tds_pickle

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

pickle_policy_snapshot

```
pickle_policy_snapshot(policy: 'PicklePolicy | None' = None) -> 'dict[str, Any]'
```

Import: staqtapp_tds.tds_pickle

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

get_default_serialization_manager

```
get_default_serialization_manager() -> 'SerializationManager'
```

Import: staqtapp_tds.serialization.manager

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

SerializationManager

```
SerializationManager(registry: 'CodecRegistry | None' = None)
```

First-class codec dispatch used by storage. Prefer directory write/read calls unless registering a custom codec or integrating an external transport.

SerializationManager.serialize

```
serialize(self, value: 'Any', fmt_id: 'int | None' = None) -> 'EncodedPayload'
```

Public operational method on this integration surface.

SerializationManager.deserialize

```
deserialize(self, raw: 'bytes', fmt_id: 'int') -> 'Any'
```

Public operational method on this integration surface.

SerializationManager.snapshot

```
snapshot(self) -> 'dict[str, Any]'
```

Return a JSON-safe snapshot.

CodecRegistry

```
CodecRegistry(codecs: 'Iterable[SerializationCodec]' = ())
```

Registry for SerializationCodec implementations.

CodecRegistry.register

```
register(self, codec: 'SerializationCodec', *, replace: 'bool' = False) -> 'None'
```

Public operational method on this integration surface.

CodecRegistry.snapshot

```
snapshot(self) -> 'dict[str, Any]'
```

Return a JSON-safe snapshot.

Pickle boundary

General Python-object storage can use the restricted pickle codec for compatibility. Treat untrusted pickle bytes as hostile input. Prefer JSON, text, raw bytes, or a custom SerializationCodec for externally supplied data.

6. Provenance, verification, telemetry, diagnostics, and recovery

6.1 Health and invariant checks

verify

```
verify(target: 'Any') -> 'HealthReport'
```

Import: staqtapp_tds.verify

Behavior: Run the TDS health verifier against a supported target and return a structured report.

estimate_pressure

```
estimate_pressure(counters: 'Mapping[str, int]', *, queue_depth: 'int' = 0, snapshot_lag: 'int' = 0, swiss_probe_pressure: 'int' = 0) -> 'PressureReport'
```

Import: staqtapp_tds.asi

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

calculate_pressure_snapshot

```
calculate_pressure_snapshot(counters: 'Mapping[str, Any]', *, native_counters: 'Mapping[str, Any] | None' = None, perform_snapshot: 'bool' = True) -> 'PressureReport'
```

Import: staqtapp_tds.pressure

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

build_recovery_plan

```
build_recovery_plan(pressure: 'Mapping[str, Any] | None', *, native_diagnostics: 'Mapping[str, Any] | None' = None, perform_snapshot: 'bool' = True) -> 'RecoveryPlan'
```

Import: staqtapp_tds.recovery

Behavior: Convert pressure/diagnostic evidence into a non-mutating recovery plan.

classify_latency

```
classify_latency(actual_ns: 'int', expected_ns: 'int', soft_limit_ns: 'int' = 0, hard_limit_ns: 'int' = 0) -> 'LatencyBucket'
```

Import: staqtapp_tds.latency

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

latency_ratio

```
latency_ratio(actual_ns: 'int', expected_ns: 'int') -> 'float'
```

Import: staqtapp_tds.latency

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

InvariantEngine

```
InvariantEngine(*, max_entries: 'int' = -1, check_latency: 'bool' = False)
```

Run structural invariants and return reports without changing stored content.

6.2 Native diagnostics

native_diagnostics_available

```
native_diagnostics_available() -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

native_diag_snapshot

```
native_diag_snapshot(*, event_limit: 'int' = 32) -> 'NativeDiagnosticSnapshot'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_reset

```
native_diag_reset() -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_set_enabled

```
native_diag_set_enabled(enabled: 'bool') -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_mark_degraded

```
native_diag_mark_degraded(degraded: 'bool' = True) -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_emit

```
native_diag_emit(event: 'DiagnosticEvent | int', value_a: 'int' = 0, value_b: 'int' = 0) -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_status_result

```
native_status_result() -> 'TDSResult'
```

Import: staqtapp_tds.native.manager

Behavior: Return a non-halting result describing native engine availability and state.

native_capabilities_result

```
native_capabilities_result() -> 'TDSResult'
```

Import: staqtapp_tds.native.manager

Behavior: Return a non-halting result containing the native capability matrix.

get_native_manager

```
get_native_manager() -> 'NativeEngineManager'
```

Import: staqtapp_tds.native.manager

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

Hot-path rule

Storage emits bounded counters/events; the diagnostic engine consumes copies. A full diagnostic ring drops diagnostic events and increments a counter rather than blocking storage.

6.3 Manifest, routing, and provenance helpers

load_manifest

```
load_manifest(folder: 'Path', *, inherit: 'bool' = True) -> 'ManifestPolicy'
```

Import: staqtapp_tds.manifest

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

write_default_manifest

```
write_default_manifest(folder: 'Path', *, overwrite: 'bool' = False) -> 'Path'
```

Import: staqtapp_tds.manifest

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

route_id_for

```
route_id_for(stamp: 'str', path: 'str' = '') -> 'int'
```

Import: staqtapp_tds.srz

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

query_requires_selector

```
query_requires_selector(**selectors) -> 'TDSResult'
```

Import: staqtapp_tds.cluster

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

7. Trace storage, provenance, and ranking

The Spiral namespace is content-neutral: TDS stores and ranks trace evidence but does not implement an external reasoning system. Ranking is deterministic, supports native/Python parity, and can return records, a complete run, or TDSResult.

```
from staqtapp_tds.spiral import NativeSpiralRankEngine, SpiralRankConfig

engine = NativeSpiralRankEngine(
    SpiralRankConfig(),
    prefer_native=True,
)
ranked = engine.rank(
    ["trace-b", "trace-a", "trace-c"],
    [0.80, 0.91, 0.91],
    confidences=[0.90, 0.95, 0.95],
    depths=[1, 3, 1],
    ages_ns=[0, 0, 0],
)
assert [r.rank for r in ranked] == [1, 2, 3]
```

create_spiral_run

```
create_spiral_run(root, run_id: 'str', *, problem: 'Any | None' = None, problem_id: 'str' = '', description: 'str' = '',
```

Import: staqtapp_tds.spiral.run

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

rank_traces

```
rank_traces(trace_ids: 'Iterable[str]', scores: 'Iterable[float]', **kwargs: 'Any') -> 'list[SpiralRankRecord]'
```

Import: staqtapp_tds.spiral.rank

Behavior: Rank trace IDs deterministically from scores and optional confidence/depth/age inputs.

rank_trace_run

```
rank_trace_run(trace_ids: 'Iterable[str]', scores: 'Iterable[float]', **kwargs: 'Any') -> 'SpiralRankRun'
```

Import: staqtapp_tds.spiral.rank

Behavior: Return deterministic ranking output from the supplied trace evidence.

rank_trace_result

```
rank_trace_result(trace_ids: 'Iterable[str]', scores: 'Iterable[float]', **kwargs: 'Any') -> 'TDSResult'
```

Import: staqtapp_tds.spiral.rank

Behavior: Return deterministic ranking output from the supplied trace evidence.

SpiralRun

```
SpiralRun(run_dir: 'Any', run_id: 'str') -> None
```

Stores one trace run under a TDS directory and maintains its manifest/provenance records.

SpiralRun metadata

```
metadata # property
```

Public operational method on this integration surface.

NativeSpiralRankEngine

```
NativeSpiralRankEngine(config: 'SpiralRankConfig | None' = None, *, prefer_native: 'bool' = True)
```

Deterministic top-N ranking engine with optional compiled scoring.

NativeSpiralRankEngine native_available

```
native_available # property
```

Public operational method on this integration surface.

NativeSpiralRankEngine score_many

```
score_many(self, scores: 'Iterable[float]', confidences: 'Iterable[float] | None' = None, depths: 'Iterable[int] | None'
```

Public operational method on this integration surface.

NativeSpiralRankEngine.rank

```
rank(self, trace_ids: 'Iterable[str]', scores: 'Iterable[float]', confidences: 'Iterable[float] | None' = None, depths:
```

Public operational method on this integration surface.

NativeSpiralRankEngine.rank_run

```
rank_run(self, trace_ids: 'Iterable[str]', scores: 'Iterable[float]', confidences: 'Iterable[float] | None' = None, dept
```

Public operational method on this integration surface.

NativeSpiralRankEngine.rank_result

```
rank_result(self, trace_ids: 'Iterable[str]', scores: 'Iterable[float]', confidences: 'Iterable[float] | None' = None, d
```

Public operational method on this integration surface.

8. CSV Suite: direct operational workflow

The CSV Suite is a layered evidence system. A normal integration does not need every layer. Choose the stopping point required by your application.

Stage	Core calls	When to stop
Import/preserve	<code>import_csv_bytes()</code> , <code>import_csv_file()</code>	You need managed CSV storage and original-byte identity.
Validate/reload/export	<code>validate_csv_artifacts()</code> , <code>reload_csv_artifacts()</code> , <code>export_original_csv()</code>	You need storage-grade artifact integrity and deterministic exports.
Transaction/recovery	<code>begin_csv_artifact_transaction()</code> , <code>commit_csv_artifact_transaction()</code> , <code>recover_csv_artifact_transaction()</code>	You need explicit staged publication and interrupted-write handling.
Scan evidence	<code>scan_csv_bytes()</code> , <code>scan_csv_row_anchors()</code> , <code>materialize_csv_scan_artifacts()</code>	You need deterministic row/anchor profiles.
Storage/native evidence	<code>commit_csv_storage_bridge_manifest()</code> , <code>commit_csv_native_storage_artifacts()</code>	You need complete storage-binding and native-storage evidence.
Kernel/Interpole	<code>commit_csv_kernel_readiness_contract_report()</code> , <code>commit_csv_interpole_timeline_ring_report()</code>	You need native parity gates and comparative readiness telemetry.
Semantic review	<code>prepare_csv_semantic_ir_candidate()</code> , <code>prepare_csv_semantic_ir_transition_batch()</code>	You need explicit, authorized, read-only semantic propositions and review lineage.

CSV invariants

The preserved original remains the source evidence. Derived artifacts are reversible. The layer avoids per-cell storage writes. Optional native scan kernels must match the Python reference path. Semantic APIs never rewrite the CSV source.

9. CSV import, validation, export, and transactions

9.1 Core import/export calls

import_csv_bytes

```
import_csv_bytes(directory: 'TSDirectory', raw: 'bytes', *, source_name: 'str' = 'csv_source.csv', encoding: 'str' = 'utf-8') -> 'bytes'
```

Import: staqtapp_tds.csv_layer.importer

Behavior: Import immutable CSV bytes and write the core CSV artifact family into a TDS directory.

import_csv_file

```
import_csv_file(directory: 'TSDirectory', path: 'str | Path', *, encoding: 'str' = 'utf-8', csv_id: 'str | None' = None) -> 'bytes'
```

Import: staqtapp_tds.csv_layer.importer

Behavior: Read a file, preserve its bytes/text identity, and write the core CSV artifact family.

load_csv_manifest

```
load_csv_manifest(directory: 'TSDirectory', csv_id: 'str') -> 'CSVImportManifest'
```

Import: staqtapp_tds.csv_layer.importer

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_artifacts

```
validate_csv_artifacts(directory: 'TSDirectory', csv_id: 'str') -> 'CSVArtifactValidationReport'
```

Import: staqtapp_tds.csv_layer.validator

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

reload_csv_artifacts

```
reload_csv_artifacts(directory: 'TSDirectory', csv_id: 'str') -> 'CSVReloadedArtifacts'
```

Import: staqtapp_tds.csv_layer.reload

Behavior: Reload the committed CSV artifact family into typed value objects.

export_original_csv

```
export_original_csv(directory: 'TSDirectory', csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Return the preserved original CSV text without canonical rewriting.

export_canonical_csv

```
export_canonical_csv(directory: 'TSDirectory', csv_id: 'str', *, lineterminator: 'str' = '\n') -> 'str'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Return a deterministic canonical CSV rendering while leaving the preserved source unchanged.

prove_original_roundtrip

```
prove_original_roundtrip(directory: 'TSDirectory', csv_id: 'str', *, overwrite: 'bool' = True) -> 'CSVRoundTripReport'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Write and return original-byte round-trip proof.

load_csv_artifact

```
load_csv_artifact(directory: 'TSDirectory', csv_id: 'str', artifact: 'str') -> 'Any'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

artifact_keys

```
artifact_keys(csv_id: 'str') -> 'dict[str, str]'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

9.2 Transaction and recovery calls

new_csv_transaction_id

```
new_csv_transaction_id() -> 'str'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

begin_csv_artifact_transaction

```
begin_csv_artifact_transaction(directory: 'TDSDirectory', raw: 'bytes', *, source_name: 'str' = 'csv_source.csv', encoding: 'str' = 'utf-8') -> 'CSVArtifactTransaction'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Stage a complete core CSV artifact family under a transaction namespace.

validate_csv_artifact_transaction

```
validate_csv_artifact_transaction(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str') -> 'CSVArtifactTransaction'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

commit_csv_artifact_transaction

```
commit_csv_artifact_transaction(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str', *, overwrite: 'bool' = False) -> 'CSVArtifactTransaction'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Validate a staged CSV transaction and publish the complete artifact family.

recover_csv_artifact_transaction

```
recover_csv_artifact_transaction(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str', *, commit_staged: 'bool' = False) -> 'CSVArtifactTransaction'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Inspect or recover an interrupted CSV artifact transaction.

detect_partial_csv_artifacts

```
detect_partial_csv_artifacts(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVArtifactTransactionReport'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Detect the requested condition or format and return deterministic evidence.

load_csv_artifact_transaction_report

```
load_csv_artifact_transaction_report(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str | None' = None) -> 'CSVArtifactTransactionReport'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

csv_artifact_transaction_keys

```
csv_artifact_transaction_keys(csv_id: 'str', transaction_id: 'str') -> 'dict[str, str]'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

validate_csv_transaction_id

```
validate_csv_transaction_id(transaction_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

```
from staqtapp_tds.csv_layer import (
    begin_csv_artifact_transaction,
    commit_csv_artifact_transaction,
    validate_csv_artifact_transaction,
)

staged = begin_csv_artifact_transaction(
    csv_dir,
    b"id,note\n1,alpha\n2,beta\n",
    source_name="events.csv",
    transaction_id="tx-events-001",
)
assert staged.ok and staged.status == "staged"

checked = validate_csv_artifact_transaction(
    csv_dir, staged.csv_id, staged.transaction_id
)
assert checked.ok

committed = commit_csv_artifact_transaction(
    csv_dir, staged.csv_id, staged.transaction_id,
    overwrite=True,
```

```
    csv_dir, staged.csv_id, staged.transaction_id
  )
  assert committed.ok and committed.status == "committed"
```

10. CSV scanning, row anchors, and artifact security

detect_csv_dialect

```
detect_csv_dialect(text: 'str', *, delimiters: 'Iterable[str]' = (',', ';', '\t', '|')) -> 'CSVDialectFingerprint'
```

Import: staqtapp_tds.csv_layer.dialect

Behavior: Detect the requested condition or format and return deterministic evidence.

dialect_to_csv_kwargs

```
dialect_to_csv_kwargs(dialect: 'CSVDialectFingerprint') -> 'dict[str, object]'
```

Import: staqtapp_tds.csv_layer.dialect

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

logical_record_offsets

```
logical_record_offsets(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8') -> 'tuple[int, ...]'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

logical_record_offsets_bytes

```
logical_record_offsets_bytes(raw: 'bytes | bytearray | memoryview', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8') -> 'tuple[int, ...]'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

build_row_offset_map

```
build_row_offset_map(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', raw: 'bytes | None' = None) -> 'dict[int, int]'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

pack_csv_row_offsets

```
pack_csv_row_offsets(row_offsets: 'Iterable[int]') -> 'bytes'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

unpack_csv_row_offsets

```
unpack_csv_row_offsets(payload: 'bytes | bytearray | memoryview') -> 'tuple[int, ...]'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

scan_csv_bytes

```
scan_csv_bytes(raw: 'BytesLike', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int | None' = None) -> 'dict[str, object]'
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Build a deterministic reference scan profile from immutable CSV bytes.

scan_csv_text

```
scan_csv_text(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int | None' = None) -> 'dict[str, object]'
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Scan immutable input and return a deterministic profile or ranking result.

validate_csv_scan_profile

```
validate_csv_scan_profile(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVScanParity'
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

scan_csv_row_anchors

```
scan_csv_row_anchors(raw: 'BytesLike', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int | None' = None) -> 'dict[str, object]'
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Build row-anchor hashes and profile information from immutable CSV bytes.

scan_csv_text_row_anchors

```
scan_csv_text_row_anchors(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Scan immutable input and return a deterministic profile or ranking result.

validate_csv_row_anchors

```
validate_csv_row_anchors(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVRowAnchorP
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_scan_artifact_keys

```
csv_scan_artifact_keys(csv_id: 'str') -> 'dict[str, str]'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

materialize_csv_scan_artifacts

```
materialize_csv_scan_artifacts(directory: 'TSDirectory', csv_id: 'str', *, include_row_anchors: 'bool' = True, chunk_si
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Persist scan profile and row-anchor evidence as bounded derived artifacts.

validate_materialized_csv_scan_artifacts

```
validate_materialized_csv_scan_artifacts(directory: 'TSDirectory', csv_id: 'str', *, require_row_anchors: 'bool' = True
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

load_csv_scan_profile

```
load_csv_scan_profile(directory: 'TSDirectory', csv_id: 'str') -> 'CSVScanProfile'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_row_anchor_profile

```
load_csv_row_anchor_profile(directory: 'TSDirectory', csv_id: 'str') -> 'CSVRowAnchorProfile'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_scan_materialization_report

```
load_csv_scan_materialization_report(directory: 'TSDirectory', csv_id: 'str') -> 'CSVScanArtifactReport'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_artifact_key

```
validate_csv_artifact_key(key: 'str', csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.security

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_artifact_security

```
validate_csv_artifact_security(directory: 'TSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False) -> '
```

Import: staqtapp_tds.csv_layer.security

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

11. CSV storage bridge, native evidence, and performance gates

csv_storage_bridge_plan

```
csv_storage_bridge_plan(csv_id: 'str', *, include_scan_artifacts: 'bool' = False, include_transaction_report: 'bool' = False)
```

Import: staqtapp_tds.csv_layer.storage_bridge

Behavior: Return the deterministic operation plan without performing storage writes.

validate_csv_storage_bridge_preflight

```
validate_csv_storage_bridge_preflight(directory: 'TDSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False)
```

Import: staqtapp_tds.csv_layer.storage_bridge

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_storage_bridge_commit

```
prepare_csv_storage_bridge_commit(directory: 'TDSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False, r)
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_storage_bridge_manifest

```
commit_csv_storage_bridge_manifest(directory: 'TDSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False,
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_storage_bridge_commit_report

```
load_csv_storage_bridge_commit_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVStorageBridgeCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_storage_bridge_commit

```
validate_csv_storage_bridge_commit(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageBridgeCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_storage_adapter_binding

```
prepare_csv_storage_adapter_binding(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterBinding'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

validate_csv_storage_adapter_binding

```
validate_csv_storage_adapter_binding(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterBinding'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_storage_adapter_replay

```
prepare_csv_storage_adapter_replay(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterReplayReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_storage_adapter_replay_report

```
commit_csv_storage_adapter_replay_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, o)
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_storage_adapter_replay_report

```
load_csv_storage_adapter_replay_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVStorageAdapterReplayReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_storage_adapter_replay

```
validate_csv_storage_adapter_replay(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterReplayReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

commit_csv_native_storage_artifacts

```
commit_csv_native_storage_artifacts(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwrite: bool = False) -> 'CSVNativeStorageCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_native_storage_commit_report

```
load_csv_native_storage_commit_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeStorageCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_native_storage_commit

```
validate_csv_native_storage_commit(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: str = 'utf-8') -> 'CSVNativeStorageCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_native_storage_revalidation

```
prepare_csv_native_storage_revalidation(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: str = 'utf-8') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_native_storage_revalidation_report

```
commit_csv_native_storage_revalidation_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: str = 'utf-8') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_native_storage_revalidation_report

```
load_csv_native_storage_revalidation_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_native_storage_revalidation

```
validate_csv_native_storage_revalidation(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: str = 'utf-8') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_kernel_readiness_contract

```
prepare_csv_kernel_readiness_contract(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVKernelReadinessContract'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_kernel_readiness_contract_report

```
commit_csv_kernel_readiness_contract_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, encoding: str = 'utf-8') -> 'CSVKernelReadinessContractReport'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_kernel_readiness_contract_report

```
load_csv_kernel_readiness_contract_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVKernelReadinessContractReport'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_kernel_readiness_contract

```
validate_csv_kernel_readiness_contract(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVKernelReadinessContract'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_native_scan_kernel_prototype

```
prepare_csv_native_scan_kernel_prototype(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, use_native_scan: bool = True) -> 'CSVNativeScanKernelPrototype'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_native_scan_kernel_prototype_report

```
commit_csv_native_scan_kernel_prototype_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVNativeScanKernelReport'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_native_scan_kernel_prototype_report

```
load_csv_native_scan_kernel_prototype_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeScanKernelReport'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_native_scan_kernel_prototype

```
validate_csv_native_scan_kernel_prototype(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVNativeScanKernelPrototype'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_native_row_anchor_kernel

```
prepare_csv_native_row_anchor_kernel(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, use_native_scan: bool = True) -> 'CSVNativeRowAnchorKernelPrototype'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_native_row_anchor_kernel_report

```
commit_csv_native_row_anchor_kernel_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVNativeRowAnchorKernelReport'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_native_row_anchor_kernel_report

```
load_csv_native_row_anchor_kernel_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeRowAnchorKernelReport'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_native_row_anchor_kernel

```
validate_csv_native_row_anchor_kernel(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVNativeRowAnchorKernelPrototype'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_kernel_performance_gates

```
prepare_csv_kernel_performance_gates(directory: 'TDSDirectory', csv_id: 'str', *, max_report_json_bytes: 'int' = 65536) -> 'CSVKernelPerformanceGates'
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_kernel_performance_gate_report

```
commit_csv_kernel_performance_gate_report(directory: 'TDSDirectory', csv_id: 'str', *, max_report_json_bytes: 'int' = 65536) -> 'CSVKernelPerformanceGateReport'
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_kernel_performance_gate_report

```
load_csv_kernel_performance_gate_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVKernelPerformanceGateReport'
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_kernel_performance_gate_report

```
validate_csv_kernel_performance_gate_report(directory: 'TDSDirectory', csv_id: 'str', *, max_report_json_bytes: 'int | None')
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract

Native mode

Use use_native=True to request the optional sidecar and force_native=True only when absence or mismatch must fail the operation. Default-safe operation retains the reference Python path.

12. CSV Interpole and centralized Browser evidence

prepare_csv_interpole_timeline

```
prepare_csv_interpole_timeline(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_interpole_timeline_report

```
commit_csv_interpole_timeline_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwrites: bool = True)
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_interpole_timeline_report

```
load_csv_interpole_timeline_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVInterpoleTimelineReport'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_interpole_timeline

```
validate_csv_interpole_timeline(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_interpole_timeline_summary

```
csv_interpole_timeline_summary(report: 'CSVInterpoleTimelineReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

prepare_csv_interpole_determinant_vector

```
prepare_csv_interpole_determinant_vector(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_interpole_determinant_vector_report

```
commit_csv_interpole_determinant_vector_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwrites: bool = True)
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_interpole_determinant_vector_report

```
load_csv_interpole_determinant_vector_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVInterpoleDeterminantVectorReport'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_interpole_determinant_vector

```
validate_csv_interpole_determinant_vector(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_interpole_determinant_vector_summary

```
csv_interpole_determinant_vector_summary(report: 'CSVInterpoleDeterminantVectorReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

prepare_csv_interpole_timeline_ring

```
prepare_csv_interpole_timeline_ring(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_interpolate_timeline_ring_report

```
commit_csv_interpolate_timeline_ring_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None,
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_interpolate_timeline_ring_report

```
load_csv_interpolate_timeline_ring_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVInterpolateTimelineRingReport'
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_interpolate_timeline_ring

```
validate_csv_interpolate_timeline_ring(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encod
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_interpolate_timeline_ring_summary

```
csv_interpolate_timeline_ring_summary(report: 'CSVInterpolateTimelineRingReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

prepare_csv_interpolate_browser_monitor_snapshot

```
prepare_csv_interpolate_browser_monitor_snapshot(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7
```

Import: staqtapp_tds.csv_layer.browser_monitor

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

validate_csv_interpolate_browser_monitor_snapshot

```
validate_csv_interpolate_browser_monitor_snapshot(snapshot: 'CSVInterpolateBrowserMonitorSnapshot | Mapping[str, Any]', *, p
```

Import: staqtapp_tds.csv_layer.browser_monitor

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

replay_csv_interpolate_browser_monitor_snapshot

```
replay_csv_interpolate_browser_monitor_snapshot(directory: 'TDSDirectory', csv_id: 'str', source_snapshot: 'CSVInterpolateBr
```

Import: staqtapp_tds.csv_layer.browser_monitor

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

csv_interpolate_browser_monitor_summary

```
csv_interpolate_browser_monitor_summary(snapshot: 'CSVInterpolateBrowserMonitorSnapshot') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.browser_monitor

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_interpolate_browser_monitor_replay_summary

```
csv_interpolate_browser_monitor_replay_summary(report: 'CSVInterpolateMonitorReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.browser_monitor

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

Interpolate reports comparative movement, determinant vectors, timeline-ring evidence, mirror feedback, and readiness pressure. They do not infer schemas, entities, or semantic truth. The Browser monitor is read-only and bounded.

13. CSV Semantic IR candidates and lifecycle review

Semantic IR starts only after an explicit caller opt-in. Declarations are caller-authored propositions bound to named evidence. TDS validates contract shape, source identity, handoff evidence, fingerprints, replay, authorization scope, and lifecycle ordering.

```
from staqtapp_tds.csv_layer import (
    CSVSemanticIRDeclaration,
    CSVSemanticIRTransitionAuthorization,
    CSVSemanticIRTransitionRequest,
    prepare_csv_semantic_ir_candidate,
    prepare_csv_semantic_ir_transition,
)

declaration = CSVSemanticIRDeclaration(
    proposition_id="customer_identifier_role",
    semantic_kind="column_role",
    subject_scope="column",
    subject_locator="index:0",
    predicate="represents",
    object_value="CustomerIdentifier",
    evidence_names=("core_artifact_integrity", "native_row_anchor_parity"),
)

candidate = prepare_csv_semantic_ir_candidate(
    csv_dir,
    manifest.csv_id,
    (declaration,),
    explicit_opt_in=True,
)

request = CSVSemanticIRTransitionRequest(
    transition_id="transition-001",
    proposition_id="customer_identifier_role",
    from_state="proposed",
    to_state="validated",
    reason="Reviewer confirmed the proposition against referenced evidence.",
    authorization=CSVSemanticIRTransitionAuthorization(
        authorization_id="auth-001",
        actor_id="reviewer-001",
        authority_scope="validate_proposition",
        authorization_reference="ticket:SEM-001",
    ),
)

lifecycle = prepare_csv_semantic_ir_transition(
    csv_dir, manifest.csv_id, candidate, request
)
```

Prerequisite

A candidate requires the complete validated Semantic IR handoff evidence chain. The short snippet focuses on object construction; build and validate the storage/native/Interpole/kernel reports first, or use an application helper that prepares that chain.

prepare_csv_semantic_ir_handoff

```
prepare_csv_semantic_ir_handoff(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, source_monitor: 'SourceMonitor' = None) -> 'CSVSemanticIRHandoffReport'
```

Import: `staqtapp_tds.csv_layer.semantic_handoff`

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

validate_csv_semantic_ir_handoff

```
validate_csv_semantic_ir_handoff(report: 'CSVSemanticIRHandoffReport | Mapping[str, Any]') -> 'CSVSemanticIRHandoffValidatedReport'
```

Import: `staqtapp_tds.csv_layer.semantic_handoff`

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_semantic_ir_handoff_fingerprint

```
csv_semantic_ir_handoff_fingerprint(report: 'CSVSemanticIRHandoffReport | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_handoff

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_handoff_summary

```
csv_semantic_ir_handoff_summary(report: 'CSVSemanticIRHandoffReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_handoff

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

prepare_csv_semantic_ir_candidate

```
prepare_csv_semantic_ir_candidate(directory: 'TSDDirectory', csv_id: 'str', declarations: 'Sequence[CSVSemanticIRDeclaration]') -> 'CSVSemanticIRCandidate | Mapping[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Create an in-memory, caller-declared Semantic IR candidate after explicit opt-in and evidence validation.

validate_csv_semantic_ir_candidate

```
validate_csv_semantic_ir_candidate(candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRValidationReport | Mapping[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

replay_csv_semantic_ir_candidate

```
replay_csv_semantic_ir_candidate(directory: 'TSDDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRCandidate | Mapping[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

csv_semantic_ir_candidate_fingerprint

```
csv_semantic_ir_candidate_fingerprint(candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_candidate_summary

```
csv_semantic_ir_candidate_summary(candidate: 'CSVSemanticIRCandidate') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_semantic_ir_replay_summary

```
csv_semantic_ir_replay_summary(report: 'CSVSemanticIRReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

prepare_csv_semantic_ir_transition

```
prepare_csv_semantic_ir_transition(directory: 'TSDDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRTransitionRecord | Mapping[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Create one authorized in-memory lifecycle transition without writing TDS artifacts.

validate_csv_semantic_ir_lifecycle

```
validate_csv_semantic_ir_lifecycle(lifecycle: 'CSVSemanticIRLifecycle | Mapping[str, Any]', *, source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRLifecycle | Mapping[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

replay_csv_semantic_ir_lifecycle

```
replay_csv_semantic_ir_lifecycle(directory: 'TSDDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRLifecycle | Mapping[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

csv_semantic_ir_transition_authorization_fingerprint

```
csv_semantic_ir_transition_authorization_fingerprint(authorization: 'CSVSemanticIRTransitionAuthorization | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_transition_fingerprint

```
csv_semantic_ir_transition_fingerprint(record: 'CSVSemanticIRTransitionRecord | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return the deterministic fingerprint for the supplied contract object.

CSV_Semantic_IR_Lifecycle_Fingerprint

```
csv_semantic_ir_lifecycle_fingerprint(lifecycle: 'CSVSemanticIRLifecycle | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return the deterministic fingerprint for the supplied contract object.

CSV_Semantic_IR_Lifecycle_Summary

```
csv_semantic_ir_lifecycle_summary(lifecycle: 'CSVSemanticIRLifecycle') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

CSV_Semantic_IR_Lifecycle_Replay_Summary

```
csv_semantic_ir_lifecycle_replay_summary(report: 'CSVSemanticIRLifecycleReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

prepare_csv_semantic_ir_transition_batch

```
prepare_csv_semantic_ir_transition_batch(directory: 'TSDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate')
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Preflight and simulate an all-or-nothing batch of authorized lifecycle transitions.

validate_csv_semantic_ir_transition_batch

```
validate_csv_semantic_ir_transition_batch(receipt: 'CSVSemanticIRBatchReceipt | Mapping[str, Any]', *, source_candidate: 'CSVSemanticIRCandidate')
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

replay_csv_semantic_ir_transition_batch

```
replay_csv_semantic_ir_transition_batch(directory: 'TSDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate')
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

CSV_Semantic_IR_Transition_Request_Fingerprint

```
csv_semantic_ir_transition_request_fingerprint(request: 'CSVSemanticIRTransitionRequest | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

CSV_Semantic_IR_Batch_Authorization_Fingerprint

```
csv_semantic_ir_batch_authorization_fingerprint(authorization: 'CSVSemanticIRBatchAuthorization | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

CSV_Semantic_IR_Transition_Batch_Fingerprint

```
csv_semantic_ir_transition_batch_fingerprint(batch: 'CSVSemanticIRTransitionBatch | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

CSV_Semantic_IR_Batch_Receipt_Fingerprint

```
csv_semantic_ir_batch_receipt_fingerprint(receipt: 'CSVSemanticIRBatchReceipt | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

CSV_Semantic_IR_Transition_Batch_Summary

```
csv_semantic_ir_transition_batch_summary(receipt: 'CSVSemanticIRBatchReceipt') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

CSV_Semantic_IR_Transition_Batch_Replay_Summary

```
csv_semantic_ir_transition_batch_replay_summary(report: 'CSVSemanticIRBatchReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

v3.5.2 state boundary

Admitted states are proposed, validated, and contested. The contract does not admit committed or superseded, and it performs no automatic transition.

14. CSV operational call index

This section lists every public function exported by `staqtapp_tds.csv_layer`, grouped by implementation module. The earlier sections explain the primary workflow; this index is for exact lookup.

14.1 browser_monitor

~~prepare_csv_interpole_browser_monitor_snapshot~~

```
prepare_csv_interpole_browser_monitor_snapshot(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

~~csv_interpole_browser_monitor_display_contract~~

```
csv_interpole_browser_monitor_display_contract() -> 'dict[str, Any]'
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

~~csv_interpole_browser_monitor_display_contract_fingerprint~~

```
csv_interpole_browser_monitor_display_contract_fingerprint() -> 'str'
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Return the deterministic fingerprint for the supplied contract object.

~~csv_interpole_browser_monitor_replay_summary~~

```
csv_interpole_browser_monitor_replay_summary(report: 'CSVInterpoleMonitorReplayReport') -> 'dict[str, Any]'
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

~~csv_interpole_browser_monitor_snapshot_fingerprint~~

```
csv_interpole_browser_monitor_snapshot_fingerprint(snapshot: 'CSVInterpoleBrowserMonitorSnapshot | Mapping[str, Any]') -
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Return the deterministic fingerprint for the supplied contract object.

~~csv_interpole_browser_monitor_summary~~

```
csv_interpole_browser_monitor_summary(snapshot: 'CSVInterpoleBrowserMonitorSnapshot') -> 'dict[str, Any]'
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

~~csv_interpole_monitor_delivery_manifest~~

```
csv_interpole_monitor_delivery_manifest(*, release_version: 'str' = '3.4.10', package_name: 'str' = 'staqtapp_tds_v3_4_1
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

~~csv_interpole_monitor_icon_registry~~

```
csv_interpole_monitor_icon_registry() -> 'dict[str, str]'
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

~~replay_csv_interpole_browser_monitor_snapshot~~

```
replay_csv_interpole_browser_monitor_snapshot(directory: 'TDSDirectory', csv_id: 'str', source_snapshot: 'CSVInterpoleBr
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

~~validate_csv_interpole_browser_monitor_snapshot~~

```
validate_csv_interpole_browser_monitor_snapshot(snapshot: 'CSVInterpoleBrowserMonitorSnapshot | Mapping[str, Any]', *, p
```

Import: `staqtapp_tds.csv_layer.browser_monitor`

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API

contract.

validate_csv_interpolate_monitor_icon_registry

```
validate_csv_interpolate_monitor_icon_registry(registry: 'Mapping[str, str] | None' = None, *, svg_payloads: 'Mapping[str, str]')
```

Import: staqtapp_tds.csv_layer.browser_monitor

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

14.2 dialect

detect_csv_dialect

```
detect_csv_dialect(text: 'str', *, delimiters: 'Iterable[str]' = (',', ';', '\t', '|')) -> 'CSVDialectFingerprint'
```

Import: staqtapp_tds.csv_layer.dialect

Behavior: Detect the requested condition or format and return deterministic evidence.

dialect_to_csv_kwargs

```
dialect_to_csv_kwargs(dialect: 'CSVDialectFingerprint') -> 'dict[str, object]'
```

Import: staqtapp_tds.csv_layer.dialect

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

14.3 exporter

export_canonical_csv

```
export_canonical_csv(directory: 'TDSDirectory', csv_id: 'str', *, lineterminator: 'str' = '\n') -> 'str'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Return a deterministic canonical CSV rendering while leaving the preserved source unchanged.

export_original_csv

```
export_original_csv(directory: 'TDSDirectory', csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Return the preserved original CSV text without canonical rewriting.

load_csv_artifact

```
load_csv_artifact(directory: 'TDSDirectory', csv_id: 'str', artifact: 'str') -> 'Any'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

prove_original_roundtrip

```
prove_original_roundtrip(directory: 'TDSDirectory', csv_id: 'str', *, overwrite: 'bool' = True) -> 'CSVRoundTripReport'
```

Import: staqtapp_tds.csv_layer.exporter

Behavior: Write and return original-byte round-trip proof.

14.4 importer

import_csv_bytes

```
import_csv_bytes(directory: 'TDSDirectory', raw: 'bytes', *, source_name: 'str' = 'csv_source.csv', encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.importer

Behavior: Import immutable CSV bytes and write the core CSV artifact family into a TDS directory.

import_csv_file

```
import_csv_file(directory: 'TDSDirectory', path: 'str | Path', *, encoding: 'str' = 'utf-8', csv_id: 'str | None' = None)
```

Import: staqtapp_tds.csv_layer.importer

Behavior: Read a file, preserve its bytes/text identity, and write the core CSV artifact family.

load_csv_manifest

```
load_csv_manifest(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVImportManifest'
```

Import: staqtapp_tds.csv_layer.importer

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

14.5 interpolate

prepare_csv_interpolate_timeline

```
prepare_csv_interpolate_timeline(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_interpolate_timeline_report

```
commit_csv_interpolate_timeline_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwrite: bool = False)
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_interpolate_timeline_report

```
load_csv_interpolate_timeline_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVInterpolateTimelineReport'
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_interpolate_timeline

```
validate_csv_interpolate_timeline(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_interpolate_determinant_vector

```
prepare_csv_interpolate_determinant_vector(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_interpolate_determinant_vector_report

```
commit_csv_interpolate_determinant_vector_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwrite: bool = False)
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_interpolate_determinant_vector_report

```
load_csv_interpolate_determinant_vector_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVInterpolateDeterminantVectorReport'
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_interpolate_determinant_vector

```
validate_csv_interpolate_determinant_vector(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_interpolate_timeline_ring

```
prepare_csv_interpolate_timeline_ring(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8')
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_interpolate_timeline_ring_report

```
commit_csv_interpolate_timeline_ring_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwrite: bool = False)
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_interpolate_timeline_ring_report

```
load_csv_interpolate_timeline_ring_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVInterpolateTimelineRingReport'
```

Import: staqtapp_tds.csv_layer.interpolate

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_interpole_timeline_ring

```
validate_csv_interpole_timeline_ring(directory: 'TSDDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encod
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_interpole_timeline_report_key

```
csv_interpole_timeline_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_interpole_timeline_summary

```
csv_interpole_timeline_summary(report: 'CSVInterpoleTimelineReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_interpole_determinant_vector_report_key

```
csv_interpole_determinant_vector_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_interpole_determinant_vector_summary

```
csv_interpole_determinant_vector_summary(report: 'CSVInterpoleDeterminantVectorReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_interpole_timeline_ring_report_key

```
csv_interpole_timeline_ring_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_interpole_timeline_ring_summary

```
csv_interpole_timeline_ring_summary(report: 'CSVInterpoleTimelineRingReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.interpole

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.6 kernel

prepare_csv_kernel_readiness_contract

```
prepare_csv_kernel_readiness_contract(directory: 'TSDDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSV
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

commit_csv_kernel_readiness_contract_report

```
commit_csv_kernel_readiness_contract_report(directory: 'TSDDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, o
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

load_csv_kernel_readiness_contract_report

```
load_csv_kernel_readiness_contract_report(directory: 'TSDDirectory', csv_id: 'str') -> 'CSVKernelReadinessReport'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_kernel_readiness_contract

```
validate_csv_kernel_readiness_contract(directory: 'TSDDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CS
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_kernel_readiness_report_key

```
csv_kernel_readiness_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_kernel_readiness_contract_summary

```
csv_kernel_readiness_contract_summary(report: 'CSVKernelReadinessReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.kernel

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.7 manifest

artifact_keys

```
artifact_keys(csv_id: 'str') -> 'dict[str, str]'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

build_manifest

```
build_manifest(*, source_name: 'str', text: 'str', raw: 'bytes', encoding: 'str', dialect: 'CSVDialectFingerprint', csv_
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

iter_rows

```
iter_rows(text: 'str', dialect: 'CSVDialectFingerprint') -> 'Iterator[list[str]]'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

row_count_and_column_count

```
row_count_and_column_count(text: 'str', dialect: 'CSVDialectFingerprint') -> 'tuple[int, int]'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

safe_csv_id

```
safe_csv_id(source_name: 'str', raw: 'bytes') -> 'str'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Validate or normalize an identifier using the public safety contract.

validate_csv_id

```
validate_csv_id(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

is_safe_csv_id

```
is_safe_csv_id(csv_id: 'str') -> 'bool'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Validate or normalize an identifier using the public safety contract.

sha256_hex

```
sha256_hex(data: 'bytes') -> 'str'
```

Import: staqtapp_tds.csv_layer.manifest

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

14.8 native_row_anchor

prepare_csv_native_row_anchor_kernel

```
prepare_csv_native_row_anchor_kernel(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, use_nati
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation

states otherwise.

load_csv_native_row_anchor_kernel_report

```
load_csv_native_row_anchor_kernel_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeRowAnchorKernelReport'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_native_row_anchor_kernel

```
validate_csv_native_row_anchor_kernel(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVNativeRowAnchorKernelReport'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

commit_csv_native_row_anchor_kernel_report

```
commit_csv_native_row_anchor_kernel_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, use_tds_artifact_contract: bool = True) -> 'CSVNativeRowAnchorKernelReport'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

csv_native_row_anchor_kernel_report_key

```
csv_native_row_anchor_kernel_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_native_row_anchor_kernel_summary

```
csv_native_row_anchor_kernel_summary(report: 'CSVNativeRowAnchorKernelReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.native_row_anchor

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.9 native_scan

prepare_csv_native_scan_kernel_prototype

```
prepare_csv_native_scan_kernel_prototype(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, use_tds_artifact_contract: bool = True) -> 'CSVNativeScanKernelPrototype'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

load_csv_native_scan_kernel_prototype_report

```
load_csv_native_scan_kernel_prototype_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeScanKernelReport'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_native_scan_kernel_prototype

```
validate_csv_native_scan_kernel_prototype(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7) -> 'CSVNativeScanKernelPrototype'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

commit_csv_native_scan_kernel_prototype_report

```
commit_csv_native_scan_kernel_prototype_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, use_tds_artifact_contract: bool = True) -> 'CSVNativeScanKernelReport'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

csv_native_scan_kernel_report_key

```
csv_native_scan_kernel_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_native_scan_kernel_summary

```
csv_native_scan_kernel_summary(report: 'CSVNativeScanKernelReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.native_scan

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.10 performance_gates

prepare_csv_kernel_performance_gates

```
prepare_csv_kernel_performance_gates(directory: 'TDSDirectory', csv_id: 'str', *, max_report_json_bytes: 'int' = 65536,
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

load_csv_kernel_performance_gate_report

```
load_csv_kernel_performance_gate_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVKernelPerformanceGateReport'
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

validate_csv_kernel_performance_gate_report

```
validate_csv_kernel_performance_gate_report(directory: 'TDSDirectory', csv_id: 'str', *, max_report_json_bytes: 'int' | None,
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

commit_csv_kernel_performance_gate_report

```
commit_csv_kernel_performance_gate_report(directory: 'TDSDirectory', csv_id: 'str', *, max_report_json_bytes: 'int' = 65536,
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

csv_kernel_performance_gate_report_key

```
csv_kernel_performance_gate_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_kernel_performance_gate_summary

```
csv_kernel_performance_gate_summary(report: 'CSVKernelPerformanceGateReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.performance_gates

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.11 reload

reload_csv_artifacts

```
reload_csv_artifacts(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVReloadedArtifacts'
```

Import: staqtapp_tds.csv_layer.reload

Behavior: Reload the committed CSV artifact family into typed value objects.

14.12 row_offsets

build_row_offset_map

```
build_row_offset_map(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', raw: 'bytes | None' = None,
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

logical_record_offsets

```
logical_record_offsets(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8') -> 'tuple[int, ...]'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

logical_record_offsets_bytes

```
logical_record_offsets_bytes(raw: 'bytes | bytearray | memoryview', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8',
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

pack_csv_row_offsets

```
pack_csv_row_offsets(row_offsets: 'Iterable[int]') -> 'bytes'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

unpack_csv_row_offsets

```
unpack_csv_row_offsets(payload: 'bytes | bytearray | memoryview') -> 'tuple[int, ...]'
```

Import: staqtapp_tds.csv_layer.row_offsets

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

14.13 scan_artifacts

csv_scan_artifact_keys

```
csv_scan_artifact_keys(csv_id: 'str') -> 'dict[str, str]'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

load_csv_row_anchor_profile

```
load_csv_row_anchor_profile(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVRowAnchorProfile'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_scan_materialization_report

```
load_csv_scan_materialization_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVScanArtifactReport'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_scan_profile

```
load_csv_scan_profile(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVScanProfile'
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

materialize_csv_scan_artifacts

```
materialize_csv_scan_artifacts(directory: 'TDSDirectory', csv_id: 'str', *, include_row_anchors: 'bool' = True, chunk_size: 'int | None' = None)
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Persist scan profile and row-anchor evidence as bounded derived artifacts.

validate_materialized_csv_scan_artifacts

```
validate_materialized_csv_scan_artifacts(directory: 'TDSDirectory', csv_id: 'str', *, require_row_anchors: 'bool' = True, chunk_size: 'int | None' = None)
```

Import: staqtapp_tds.csv_layer.scan_artifacts

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

14.14 scanner

scan_csv_bytes

```
scan_csv_bytes(raw: 'BytesLike', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int | None' = None)
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Build a deterministic reference scan profile from immutable CSV bytes.

scan_csv_text

```
scan_csv_text(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int | None' = None)
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Scan immutable input and return a deterministic profile or ranking result.

validate_csv_row_anchors

```
validate_csv_row_anchors(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVRowAnchorProfile'
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

contract.

scan_csv_text_row_anchors

```
scan_csv_text_row_anchors(text: 'str', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Scan immutable input and return a deterministic profile or ranking result.

scan_csv_row_anchors

```
scan_csv_row_anchors(raw: 'BytesLike', dialect: 'CSVDialectFingerprint', *, encoding: 'str' = 'utf-8', chunk_size: 'int
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Build row-anchor hashes and profile information from immutable CSV bytes.

validate_csv_scan_profile

```
validate_csv_scan_profile(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVScanParit
```

Import: staqtapp_tds.csv_layer.scanner

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

14.15 security

validate_csv_artifact_security

```
validate_csv_artifact_security(directory: 'TDSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False) -> '
```

Import: staqtapp_tds.csv_layer.security

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_artifact_key

```
validate_csv_artifact_key(key: 'str', csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.security

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

14.16 semantic_handoff

prepare_csv_semantic_ir_handoff

```
prepare_csv_semantic_ir_handoff(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = 7, source_monito
```

Import: staqtapp_tds.csv_layer.semantic_handoff

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

validate_csv_semantic_ir_handoff

```
validate_csv_semantic_ir_handoff(report: 'CSVSemanticIRHandoffReport | Mapping[str, Any]') -> 'CSVSemanticIRHandoffValid
```

Import: staqtapp_tds.csv_layer.semantic_handoff

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_semantic_ir_handoff_fingerprint

```
csv_semantic_ir_handoff_fingerprint(report: 'CSVSemanticIRHandoffReport | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_handoff

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_handoff_summary

```
csv_semantic_ir_handoff_summary(report: 'CSVSemanticIRHandoffReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_handoff

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.17 semantic_ir

csv_semantic_ir_declaration_fingerprint

```
csv_semantic_ir_declaration_fingerprint(declaration: 'CSVSemanticIRDeclaration | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return the deterministic fingerprint for the supplied contract object.

prepare_csv_semantic_ir_candidate

```
prepare_csv_semantic_ir_candidate(directory: 'TDSDirectory', csv_id: 'str', declarations: 'Sequence[CSVSemanticIRDeclaration]') -> 'CSVSemanticIRCandidate'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Create an in-memory, caller-declared Semantic IR candidate after explicit opt-in and evidence validation.

validate_csv_semantic_ir_candidate

```
validate_csv_semantic_ir_candidate(candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRValidationResult'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_semantic_ir_candidate_fingerprint

```
csv_semantic_ir_candidate_fingerprint(candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return the deterministic fingerprint for the supplied contract object.

replay_csv_semantic_ir_candidate

```
replay_csv_semantic_ir_candidate(directory: 'TDSDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRCandidate'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

csv_semantic_ir_candidate_summary

```
csv_semantic_ir_candidate_summary(candidate: 'CSVSemanticIRCandidate') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_semantic_ir_replay_summary

```
csv_semantic_ir_replay_summary(report: 'CSVSemanticIRReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.18 semantic_ir_lifecycle

csv_semantic_ir_transition_authorization_fingerprint

```
csv_semantic_ir_transition_authorization_fingerprint(authorization: 'CSVSemanticIRTransitionAuthorization | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_transition_fingerprint

```
csv_semantic_ir_transition_fingerprint(record: 'CSVSemanticIRTransitionRecord | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return the deterministic fingerprint for the supplied contract object.

prepare_csv_semantic_ir_transition

```
prepare_csv_semantic_ir_transition(directory: 'TDSDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'CSVSemanticIRTransitionRecord'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Create one authorized in-memory lifecycle transition without writing TDS artifacts.

validate_csv_semantic_ir_lifecycle

```
validate_csv_semantic_ir_lifecycle(lifecycle: 'CSVSemanticIRLifecycle | Mapping[str, Any]', *, source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'bool'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

csv_semantic_ir_lifecycle_fingerprint

```
csv_semantic_ir_lifecycle_fingerprint(lifecycle: 'CSVSemanticIRLifecycle | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return the deterministic fingerprint for the supplied contract object.

replay_csv_semantic_ir_lifecycle

```
replay_csv_semantic_ir_lifecycle(directory: 'TSDDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

csv_semantic_ir_lifecycle_summary

```
csv_semantic_ir_lifecycle_summary(lifecycle: 'CSVSemanticIRLifecycle') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_semantic_ir_lifecycle_replay_summary

```
csv_semantic_ir_lifecycle_replay_summary(report: 'CSVSemanticIRLifecycleReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.19 semantic_ir_lifecycle_batch

csv_semantic_ir_transition_request_fingerprint

```
csv_semantic_ir_transition_request_fingerprint(request: 'CSVSemanticIRTransitionRequest | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_batch_authorization_fingerprint

```
csv_semantic_ir_batch_authorization_fingerprint(authorization: 'CSVSemanticIRBatchAuthorization | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_transition_batch_fingerprint

```
csv_semantic_ir_transition_batch_fingerprint(batch: 'CSVSemanticIRTransitionBatch | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

csv_semantic_ir_batch_receipt_fingerprint

```
csv_semantic_ir_batch_receipt_fingerprint(receipt: 'CSVSemanticIRBatchReceipt | Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return the deterministic fingerprint for the supplied contract object.

prepare_csv_semantic_ir_transition_batch

```
prepare_csv_semantic_ir_transition_batch(directory: 'TSDDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Preflight and simulate an all-or-nothing batch of authorized lifecycle transitions.

validate_csv_semantic_ir_transition_batch

```
validate_csv_semantic_ir_transition_batch(receipt: 'CSVSemanticIRBatchReceipt | Mapping[str, Any]', *, source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

replay_csv_semantic_ir_transition_batch

```
replay_csv_semantic_ir_transition_batch(directory: 'TSDDirectory', csv_id: 'str', source_candidate: 'CSVSemanticIRCandidate | Mapping[str, Any]') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Recompute the current result and compare it with the supplied source object to detect drift or tampering.

csv_semantic_ir_transition_batch_summary

```
csv_semantic_ir_transition_batch_summary(receipt: 'CSVSemanticIRBatchReceipt') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_semantic_ir_transition_batch_replay_summary

```
csv_semantic_ir_transition_batch_replay_summary(report: 'CSVSemanticIRBatchReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.semantic_ir_lifecycle_batch

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

14.20 storage_adapter

validate_csv_storage_bridge_commit

```
validate_csv_storage_bridge_commit(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageBridgeCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_storage_adapter_binding

```
validate_csv_storage_adapter_binding(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterBindingReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_storage_adapter_replay

```
validate_csv_storage_adapter_replay(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterReplayReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_native_storage_commit

```
validate_csv_native_storage_commit(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8') -> 'CSVNativeStorageCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_native_storage_revalidation

```
validate_csv_native_storage_revalidation(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

prepare_csv_storage_bridge_commit

```
prepare_csv_storage_bridge_commit(directory: 'TSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False, report_type: 'str' = 'read-only') -> 'CSVStorageBridgeCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

prepare_csv_storage_adapter_binding

```
prepare_csv_storage_adapter_binding(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterBindingReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

prepare_csv_storage_adapter_replay

```
prepare_csv_storage_adapter_replay(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None) -> 'CSVStorageAdapterReplayReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

prepare_csv_native_storage_revalidation

```
prepare_csv_native_storage_revalidation(directory: 'TSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, encoding: 'str' = 'utf-8') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Build a deterministic read-only report or candidate from current evidence; does not publish the result unless the specific implementation states otherwise.

load_csv_storage_bridge_commit_report

```
load_csv_storage_bridge_commit_report(directory: 'TSDirectory', csv_id: 'str') -> 'CSVStorageBridgeCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_storage_adapter_replay_report

```
load_csv_storage_adapter_replay_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVStorageAdapterReplayReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_native_storage_commit_report

```
load_csv_native_storage_commit_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeStorageCommitReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

load_csv_native_storage_revalidation_report

```
load_csv_native_storage_revalidation_report(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVNativeStorageRevalidationReport'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

csv_storage_bridge_commit_summary

```
csv_storage_bridge_commit_summary(report: 'CSVStorageBridgeCommitReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_storage_adapter_binding_summary

```
csv_storage_adapter_binding_summary(report: 'CSVStorageAdapterBindingReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_storage_adapter_replay_summary

```
csv_storage_adapter_replay_summary(report: 'CSVStorageAdapterReplayReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_native_storage_commit_summary

```
csv_native_storage_commit_summary(report: 'CSVNativeStorageCommitReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_native_storage_revalidation_summary

```
csv_native_storage_revalidation_summary(report: 'CSVNativeStorageRevalidationReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

csv_storage_bridge_commit_report_key

```
csv_storage_bridge_commit_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_storage_adapter_replay_report_key

```
csv_storage_adapter_replay_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_native_storage_commit_report_key

```
csv_native_storage_commit_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

csv_native_storage_revalidation_report_key

```
csv_native_storage_revalidation_report_key(csv_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

commit_csv_storage_bridge_manifest

```
commit_csv_storage_bridge_manifest(directory: 'TDSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = False,
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

commit_csv_storage_adapter_replay_report

```
commit_csv_storage_adapter_replay_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, o
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

commit_csv_native_storage_artifacts

```
commit_csv_native_storage_artifacts(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = None, overwr
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

commit_csv_native_storage_revalidation_report

```
commit_csv_native_storage_revalidation_report(directory: 'TDSDirectory', csv_id: 'str', *, chunk_size: 'int | None' = No
```

Import: staqtapp_tds.csv_layer.storage_adapter

Behavior: Validate current prerequisites and persist the resulting report/artifact using the TDS artifact contract.

14.21 storage_bridge

csv_storage_bridge_plan

```
csv_storage_bridge_plan(csv_id: 'str', *, include_scan_artifacts: 'bool' = False, include_transaction_report: 'bool' = F
```

Import: staqtapp_tds.csv_layer.storage_bridge

Behavior: Return the deterministic operation plan without performing storage writes.

csv_storage_bridge_preflight_summary

```
csv_storage_bridge_preflight_summary(report: 'CSVStorageBridgePreflightReport') -> 'dict[str, Any]'
```

Import: staqtapp_tds.csv_layer.storage_bridge

Behavior: Return a compact JSON-safe summary suitable for logs, Browser payloads, or external reporting.

validate_csv_storage_bridge_preflight

```
validate_csv_storage_bridge_preflight(directory: 'TDSDirectory', csv_id: 'str', *, include_scan_artifacts: 'bool' = Fals
```

Import: staqtapp_tds.csv_layer.storage_bridge

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

14.22 transaction

begin_csv_artifact_transaction

```
begin_csv_artifact_transaction(directory: 'TDSDirectory', raw: 'bytes', *, source_name: 'str' = 'csv_source.csv', encodi
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Stage a complete core CSV artifact family under a transaction namespace.

commit_csv_artifact_transaction

```
commit_csv_artifact_transaction(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str', *, overwrite: 'bool' =
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Validate a staged CSV transaction and publish the complete artifact family.

csv_artifact_transaction_keys

```
csv_artifact_transaction_keys(csv_id: 'str', transaction_id: 'str') -> 'dict[str, str]'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Return deterministic TDS artifact key name(s) for the requested object.

detect_partial_csv_artifacts

```
detect_partial_csv_artifacts(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVArtifactTransactionReport'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Detect the requested condition or format and return deterministic evidence.

load_csv_artifact_transaction_report

```
load_csv_artifact_transaction_report(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str | None' = None) -> 'str'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

new_csv_transaction_id

```
new_csv_transaction_id() -> 'str'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

recover_csv_artifact_transaction

```
recover_csv_artifact_transaction(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str', *, commit_staged: 'bool')
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Inspect or recover an interrupted CSV artifact transaction.

validate_csv_artifact_transaction

```
validate_csv_artifact_transaction(directory: 'TDSDirectory', csv_id: 'str', transaction_id: 'str') -> 'CSVArtifactTransaction'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

validate_csv_transaction_id

```
validate_csv_transaction_id(transaction_id: 'str') -> 'str'
```

Import: staqtapp_tds.csv_layer.transaction

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

14.23 validator

validate_csv_artifacts

```
validate_csv_artifacts(directory: 'TDSDirectory', csv_id: 'str') -> 'CSVArtifactValidationReport'
```

Import: staqtapp_tds.csv_layer.validator

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

15. Driver TDDL, bytecode, Foundry, and Runtime Manager

The Driver subsystem uses a closed deterministic grammar and bytecode package. Driver Foundry can validate, compile, audit, test, and submit candidate evidence. It cannot sign or activate drivers. Execution authority remains in the Runtime Manager and Registry policy chain.

```
from staqtapp_tds.drivers import DriverFoundry, DriverVMRuntime

source = """
driver SearchPolicies v1

manifest:
  kind = "search"
  description = "Find policy records"
  safety = "bounded"

requires:
  capability registry.scan
  capability manifest.read

limits:
  max_scan = 1000
  max_depth = 4
  timeout_ms = 250

program:
  SCAN scope=".tds" recursive=true limit=1000 depth=4
  READ target="manifest"
  MATCH field="manifest.kind" eq="policy"
  EXTRACT from="manifest" fields=["policy_id", "version"]
  EMIT mode="list" limit=10
  HALT

evolution:
  deny external_io
  max_delta = 1
"""

foundry = DriverFoundry()
built = foundry.compile_driver(source)
if built.ok:
    vm = DriverVMRuntime(max_cost=100_000)
    vm.load(built.package)
    records = [
        {"manifest": {"kind": "policy", "policy_id": "p1", "version": 1}}
    ]
    result = vm.execute({"records": records})
```

parse_tddl

```
parse_tddl(source: 'str') -> 'TDDLProgram'
```

Import: staqtapp_tds.drivers.tddl

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

validate_tddl

```
validate_tddl(program: 'TDDLProgram') -> 'None'
```

Import: staqtapp_tds.drivers.tddl

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

instruction_specs

```
instruction_specs() -> 'Mapping[str, InstructionSpec]'
```

Import: staqtapp_tds.drivers.tddl

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

compile_tddl

```
compile_tddl(source_or_program: 'str | TDDLProgram') -> 'BytecodePackage'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Validate TDDL source and compile it into a deterministic bytecode package.

compile_program

```
compile_program(program: 'TDDLProgram', *, source_material: 'str | Mapping[str, Any] | None' = None) -> 'BytecodePackage'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

validate_bytecode_package

```
validate_bytecode_package(package: 'BytecodePackage') -> 'None'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

decompile_to_ir

```
decompile_to_ir(package: 'BytecodePackage') -> 'TDDLProgram'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

opcode_table

```
opcode_table() -> 'Mapping[str, Mapping[str, Any]]'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

audit_vm_contract

```
audit_vm_contract(package: 'BytecodePackage') -> 'None'
```

Import: staqtapp_tds.drivers.audit

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

vm_contract_table

```
vm_contract_table() -> 'Mapping[str, Mapping[str, Any]]'
```

Import: staqtapp_tds.drivers.audit

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

sign_payload

```
sign_payload(payload: 'bytes', *, signer: 'str', secret: 'bytes') -> 'str'
```

Import: staqtapp_tds.drivers.signature

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

verify_signature

```
verify_signature(payload: 'bytes', signature: 'str', *, signer: 'str', secret: 'bytes') -> 'bool'
```

Import: staqtapp_tds.drivers.signature

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

DriverFoundry

```
DriverFoundry(*, policy: 'DriverFoundryPolicy | None' = None) -> 'None'
```

DriverFoundry.capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverFoundry.validate_driver

```
validate_driver(self, source: 'str') -> 'DriverFoundryResult'
```

Public operational method on this integration surface.

DriverFoundry.compile_driver

```
compile_driver(self, source: 'str') -> 'DriverFoundryResult'
```

Public operational method on this integration surface.

DriverFoundry.audit_driver

```
audit_driver(self, package: 'BytecodePackage') -> 'DriverFoundryResult'
```

Public operational method on this integration surface.

DriverFoundry.test_driver

```
test_driver(self, package: 'BytecodePackage', fixtures: 'Mapping[str, Any]') -> 'DriverFoundryResult'
```

Public operational method on this integration surface.

DriverFoundry.propose_driver

```
propose_driver(self, source: 'str', *, fixtures: 'Mapping[str, Any] | None' = None) -> 'DriverFoundryResult'
```

Validate, compile, audit, and optionally fixture-test a TDDL driver proposal in one non-authoritative flow.

DriverFoundry.submit_candidate

```
submit_candidate(self, package: 'BytecodePackage', *, registry: 'DriverRegistry', vm_result: 'DriverVMResult | None' = None)
```

Submit the request through the existing policy/authority layer; does not bypass authority boundaries.

DriverVMRuntime

```
DriverVMRuntime(*, max_instructions: 'int' = 1024, max_cost: 'int' = 100000) -> 'None'
```

DriverVMRuntime.load

```
load(self, package: 'BytecodePackage') -> 'VMLoadedPackage'
```

Load or prepare an object for later execution.

DriverVMRuntime.execute

```
execute(self, inputs: 'Mapping[str, Any] | None' = None) -> 'DriverVMResult'
```

Execute the currently loaded deterministic bytecode package against a bounded input snapshot.

DriverVMSkeleton

```
DriverVMSkeleton(*, max_instructions: 'int' = 1024, max_cost: 'int' = 100000) -> 'None'
```

Audit-only VM skeleton. execute() intentionally returns execution-disabled evidence.

DriverVMSkeleton.load

```
load(self, package: 'BytecodePackage') -> 'VMLoadedPackage'
```

Load or prepare an object for later execution.

DriverVMSkeleton.execute

```
execute(self, _inputs: 'Mapping[str, Any] | None' = None) -> 'DriverVMResult'
```

Execute the bounded operation.

DriverRuntimeManager

```
DriverRuntimeManager(*, policy: 'RuntimeManagerPolicy | None' = None) -> 'None'
```

DriverRuntimeManager.execute_package

```
execute_package(self, package: 'BytecodePackage', fixtures: 'Mapping[str, Any] | None', *, registry: 'DriverRegistry | None' = None)
```

Run a package through the controlled Runtime Manager and return complete execution evidence.

DriverRuntimeManager.run

```
run(self, package: 'BytecodePackage', fixtures: 'Mapping[str, Any] | None', *, registry: 'DriverRegistry | None' = None)
```

Public operational method on this integration surface.

DriverRegistry

```
DriverRegistry(signature_policy: 'SignaturePolicy | None' = None) -> 'None'
```

DriverRegistry.add_candidate

```
add_candidate(self, manifest: 'DriverManifest', *, test_report_hash: 'str | None' = None) -> 'DriverRecord'
```

Public operational method on this integration surface.

DriverRegistry.require

```
require(self, driver_id: 'str') -> 'DriverRecord'
```

Public operational method on this integration surface.

DriverRegistry.attach_signature

```
attach_signature(self, driver_id: 'str', signature: 'str') -> 'DriverRecord'
```

Public operational method on this integration surface.

DriverRegistry.approve

```
approve(self, driver_id: 'str') -> 'DriverRecord'
```

Public operational method on this integration surface.

DriverRegistry.activate

```
activate(self, driver_id: 'str') -> 'DriverRecord'
```

Public operational method on this integration surface.

DriverRegistry.retire

```
retire(self, driver_id: 'str') -> 'DriverRecord'
```

Public operational method on this integration surface.

DriverRegistry.revoke

```
revoke(self, driver_id: 'str') -> 'DriverRecord'
```

Public operational method on this integration surface.

Authority order

Candidate submission, signature attachment, approval, and activation are separate operations. Do not collapse them into a single AI-generated action. Foundry and Studio intentionally lack direct sign/activate methods.

16. Driver regression, review, evidence, and performance

DriverRegressionHarness

```
DriverRegressionHarness(*, runtime_manager: 'DriverRuntimeManager | None' = None, max_cases: 'int' = 256) -> 'None'
```

DriverRegressionHarness capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverRegressionHarness run

```
run(self, package: 'BytecodePackage', cases: 'Sequence[DriverFixtureCase | Mapping[str, Any]]', *, registry: 'DriverRegistry')
```

Public operational method on this integration surface.

DriverRegressionHarness run_package

```
run_package(self, package: 'BytecodePackage', cases: 'Sequence[DriverFixtureCase | Mapping[str, Any]]', *, registry: 'DriverRegistry')
```

Run deterministic fixture cases and return a regression report.

DriverBatchReviewBoard

```
DriverBatchReviewBoard(*, policy: 'BatchReviewPolicy | None' = None) -> 'None'
```

DriverBatchReviewBoard capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverBatchReviewBoard review

```
review(self, reports: 'Sequence[DriverRegressionReport | DriverReviewItem | Mapping[str, Any]]', *, reviewer_id: 'str' = None)
```

Public operational method on this integration surface.

DriverBatchReviewBoard review_reports

```
review_reports(self, reports: 'Sequence[DriverRegressionReport | DriverReviewItem | Mapping[str, Any]]', *, reviewer_id: 'str' = None)
```

Review regression reports and produce a bounded batch decision; registry mutation is explicit and optional.

EvidenceBundleExporter

```
EvidenceBundleExporter(*, export_format: 'EvidenceExportFormat | str' = ) -> 'None'
```

EvidenceBundleExporter capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

EvidenceBundleExporter export

```
export(self, batch_report: 'DriverBatchReviewReport', *, regression_reports: 'Sequence[DriverRegressionReport] | Mapping[str, Any]')
```

Public operational method on this integration surface.

EvidenceBundleExporter export_batch_review

```
export_batch_review(self, batch_report: 'DriverBatchReviewReport', *, regression_reports: 'Sequence[DriverRegressionReport] | Mapping[str, Any]')
```

Create an immutable JSON-safe review/evidence bundle.

EvidenceBundleExporter verify

```
verify(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str') -> 'EvidenceIntegrityStatus'
```

Verify integrity and return structured status.

EvidenceBundleExporter verify_bundle

```
verify_bundle(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str') -> 'EvidenceIntegrityStatus'
```

Verify integrity and return structured status.

DriverVMPerformanceHarness

```
DriverVMPerformanceHarness(*, policy: 'DriverVMPerformancePolicy | None' = None, runtime_manager_policy: 'RuntimeManagerPolicy | None' = None)
```

Opt-in performance evidence. Normal DriverVMRuntime execution does not benchmark itself.

DriverVMPerformanceHarness capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverVMPerformanceHarness run_package

```
run_package(self, package: 'BytecodePackage', fixtures: 'Mapping[str, Any] | None') -> 'DriverVMPerformanceReport'
```

Public operational method on this integration surface.

DriverVMPerformanceHarness run

```
run(self, package: 'BytecodePackage', fixtures: 'Mapping[str, Any] | None') -> 'DriverVMPerformanceReport'
```

Public operational method on this integration surface.

runtime_fixture_hash

```
runtime_fixture_hash(fixtures: 'Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.drivers.regression

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

run_studio_quick_test

```
run_studio_quick_test(source: 'str', *, signer: 'str' = 'studio-admin', secret: 'bytes' = b'local-driver-studio-secret')
```

Import: staqtapp_tds.drivers.studio

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_instruction_reference

```
studio_instruction_reference() -> 'Mapping[str, Mapping[str, object]]'
```

Import: staqtapp_tds.drivers.studio

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

foundry_capability_matrix

```
foundry_capability_matrix(policy: 'DriverFoundryPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.foundry

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

runtime_manager_capability_matrix

```
runtime_manager_capability_matrix(policy: 'RuntimeManagerPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.runtime_manager

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

regression_harness_capability_matrix

```
regression_harness_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.regression

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

batch_review_capability_matrix

```
batch_review_capability_matrix(policy: 'BatchReviewPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.review

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

evidence_export_capability_matrix

```
evidence_export_capability_matrix(export_format: 'EvidenceExportFormat | str' = ) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.evidence

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

driver_vm_performance_capability_matrix

```
driver_vm_performance_capability_matrix(policy: 'DriverVMPerformancePolicy | None' = None, *, native_runner_available: 'bool')
```

Import: staqtapp_tds.drivers.performance

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

driver_vm_performance_enabled

```
driver_vm_performance_enabled(env: 'Mapping[str, str] | None' = None) -> 'bool'
```

Import: `staqtapp_tds.drivers.performance`

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

17. Driver Studio and PyQt5 integration

Driver Studio has two useful programming surfaces: the headless evidence/view-model APIs and the optional PyQt5 window. Headless modules import safely without PyQt5. Studio may prepare proposals and route review requests, but it remains outside direct approval, signing, activation, Registry mutation, storage writes, and Driver VM execution.

DriverStudioReadOnlyConsole

```
DriverStudioReadOnlyConsole(*, exporter: 'EvidenceBundleExporter | None' = None) -> 'None'
```

DriverStudioReadOnlyConsole capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverStudioReadOnlyConsole open_bundle

```
open_bundle(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str', *, selected_driver_id: 'str | None' = None)
```

Public operational method on this integration surface.

DriverStudioReadOnlyConsole load_bundle

```
load_bundle(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str', *, selected_driver_id: 'str | None' = None)
```

Public operational method on this integration surface.

DriverStudioReadOnlyConsole render

```
render(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str', *, selected_driver_id: 'str | None' = None) -> 'D'
```

Public operational method on this integration surface.

DriverStudioManualProposalBuilder

```
DriverStudioManualProposalBuilder(*, foundry: 'DriverFoundry | None' = None) -> 'None'
```

DriverStudioManualProposalBuilder capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverStudioManualProposalBuilder build_source

```
build_source(self, task: 'StudioManualDriverTask') -> 'str'
```

Public operational method on this integration surface.

DriverStudioManualProposalBuilder preview_task

```
preview_task(self, task: 'StudioManualDriverTask') -> 'StudioManualProposalPreview'
```

Build a non-mutating preview suitable for review or UI display.

DriverStudioManualProposalBuilder propose_task

```
propose_task(self, task: 'StudioManualDriverTask', *, fixtures: 'Mapping[str, Any] | None' = None) -> 'StudioManualPropo'
```

Prepare a proposal and return structured evidence; does not grant approval or activation authority.

DriverStudioAdminReviewActions

```
DriverStudioAdminReviewActions(*, policy: 'StudioReviewSubmissionPolicy | None' = None, readonly_console: 'DriverStudioR'
```

DriverStudioAdminReviewActions capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

DriverStudioAdminReviewActions submit

```
submit(self, console_or_bundle: 'DriverStudioConsoleSnapshot | DriverEvidenceBundle | Mapping[str, Any] | str', actions:
```

Public operational method on this integration surface.

DriverStudioAdminReviewActions submit_actions

```
submit_actions(self, console_or_bundle: 'DriverStudioConsoleSnapshot | DriverEvidenceBundle | Mapping[str, Any] | str',
```

Submit the request through the existing policy/authority layer; does not bypass authority boundaries.

DriverStudioAdminReviewActions route

```
route(self, console_or_bundle: 'DriverStudioConsoleSnapshot | DriverEvidenceBundle | Mapping[str, Any] | str', actions:
```

Public operational method on this integration surface.

StudioQtBridge

```
StudioQtBridge(*, readonly_console: 'DriverStudioReadOnlyConsole | None' = None, action_layer: 'DriverStudioAdminReviewA
```

StudioQtBridge capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

StudioQtBridge shell_state

```
shell_state(self) -> 'StudioQtShellState'
```

Public operational method on this integration surface.

StudioQtBridge load_bundle

```
load_bundle(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str', *, selected_driver_id: 'str | None' = None)
```

Hydrate the headless/PyQt5 Studio shell from a read-only evidence bundle.

StudioQtBridge select_driver

```
select_driver(self, driver_id: 'str | None') -> 'StudioQtShellState'
```

Public operational method on this integration surface.

StudioQtBridge hydrated_shell_state

```
hydrated_shell_state(self)
```

Public operational method on this integration surface.

StudioQtBridge hydrate_panel

```
hydrate_panel(self, kind: 'StudioPanelKind | str')
```

Public operational method on this integration surface.

StudioQtBridge build_action_request

```
build_action_request(self, driver_id: 'str', action: 'ReviewAction | str', *, reviewer_id: 'str' = 'studio-admin', ratio
```

Public operational method on this integration surface.

StudioQtBridge submit_review_action

```
submit_review_action(self, request: 'StudioReviewActionRequest | Mapping[str, Any]', *, submitted_at: 'str' = 'undated')
```

Submit the request through the existing policy/authority layer; does not bypass authority boundaries.

StudioQtBridge preview_manual_driver_task

```
preview_manual_driver_task(self, task: 'StudioManualDriverTask') -> 'StudioManualProposalPreview'
```

Build a non-mutating preview suitable for review or UI display.

StudioQtBridge propose_manual_driver_task

```
propose_manual_driver_task(self, task: 'StudioManualDriverTask', *, fixtures: 'Mapping[str, Any] | None' = None) -> 'Stu
```

Prepare a proposal and return structured evidence; does not grant approval or activation authority.

StudioQtBridge live_event_bridge

```
live_event_bridge(self, *, max_events: 'int' = 256)
```

Public operational method on this integration surface.

StudioQtBridge live_panel_runtime

```
live_panel_runtime(self, *, max_events: 'int' = 256)
```

Public operational method on this integration surface.

StudioQtBridge review_workflow_console

```
review_workflow_console(self, *, max_events: 'int' = 256)
```

Public operational method on this integration surface.

StudioQtBridge evidence_timeline

```
evidence_timeline(self, *, max_events: 'int' = 256)
```

Public operational method on this integration surface.

StudioOtbBridge risk_intelligence_cards

```
risk_intelligence_cards(self, *, max_events: 'int' = 256)
```

Public operational method on this integration surface.

StudioOtbBridge manual_builder_ui_runtime

```
manual_builder_ui_runtime(self, *, max_events: 'int' = 256)
```

Public operational method on this integration surface.

StudioOtbBridge export_audit_console

```
export_audit_console(self, *, max_events: 'int' = 256)
```

Prepare or export immutable evidence in the selected format.

StudioOtbBridge export_integrity_workflow

```
export_integrity_workflow(self, *, max_events: 'int' = 256)
```

Prepare or export immutable evidence in the selected format.

StudioOtbBridge visual_quality_review

```
visual_quality_review(self)
```

Public operational method on this integration surface.

StudioLivePanelRuntime

```
StudioLivePanelRuntime(*, event_bridge: 'StudioCockpitEventBridge | None' = None, max_events: 'int' = 256, contracts: 'S
```

StudioLivePanelRuntime capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

StudioLivePanelRuntime current_state

```
current_state(self, *, include_packets: 'bool' = False) -> 'StudioPanelRuntimeState'
```

Public operational method on this integration surface.

StudioLivePanelRuntime consume

```
consume(self, *, include_packets: 'bool' = True) -> 'StudioPanelRuntimeState'
```

Public operational method on this integration surface.

StudioLivePanelRuntime load_bundle

```
load_bundle(self, bundle: 'DriverEvidenceBundle | Mapping[str, Any] | str', *, selected_driver_id: 'str | None' = None,
```

Public operational method on this integration surface.

StudioLivePanelRuntime select_driver

```
select_driver(self, driver_id: 'str | None', *, timestamp: 'str' = 'undated') -> 'StudioPanelRuntimeState'
```

Public operational method on this integration surface.

StudioLivePanelRuntime refresh

```
refresh(self, *, reason: 'str' = 'manual refresh', timestamp: 'str' = 'undated') -> 'StudioPanelRuntimeState'
```

Refresh the snapshot/view model from copied evidence without becoming storage authority.

StudioLivePanelRuntime refresh_panel

```
refresh_panel(self, kind: 'StudioPanelKind | str', *, timestamp: 'str' = 'undated') -> 'StudioPanelRuntimeState'
```

Refresh the snapshot/view model from copied evidence without becoming storage authority.

StudioLivePanelRuntime submit_review_action

```
submit_review_action(self, request: 'StudioReviewActionRequest | Mapping[str, Any]', *, submitted_at: 'str' = 'undated')
```

Submit the request through the existing policy/authority layer; does not bypass authority boundaries.

StudioLivePanelRuntime preview_manual_driver_task

```
preview_manual_driver_task(self, task: 'StudioManualDriverTask', *, timestamp: 'str' = 'undated') -> 'tuple[StudioManual
```

Build a non-mutating preview suitable for review or UI display.

StudioLivePanelRuntime propose_manual_driver_task

```
propose_manual_driver_task(self, task: 'StudioManualDriverTask', *, fixtures: 'Mapping[str, Any] | None' = None, timesta
```

Prepare a proposal and return structured evidence; does not grant approval or activation authority.

StudioLivePanelRuntime signal_payload

```
signal_payload(self) -> 'Mapping[str, Any]'
```

Public operational method on this integration surface.

StudioOperationalStressHarness

```
StudioOperationalStressHarness(*, runtime: 'StudioLivePanelRuntime | None' = None, admin_control: 'AdminControl | None'
```

StudioOperationalStressHarness capability_matrix

```
capability_matrix(self) -> 'Mapping[str, bool]'
```

Return the explicit allowed/denied capability map.

StudioOperationalStressHarness run

```
run(self, *, iterations: 'int' = 32, browser_polls: 'int | None' = None, studio_refreshes: 'int | None' = None, include_
```

Public operational method on this integration surface.

StudioOperationalStressHarness run_scenario

```
run_scenario(self, scenario: 'StudioOperationalStressScenario | str', *, iterations: 'int' = 32) -> 'StudioOperationalSt
```

Public operational method on this integration surface.

StudioOperationalStressHarness run_scenario_matrix

```
run_scenario_matrix(self, scenarios: 'tuple[StudioOperationalStressScenario | str, ...] | list[StudioOperationalStressSc
```

Execute the named Browser/Studio/persistence stress scenarios and return structured evidence.

pyqt5_available

```
pyqt5_available() -> 'bool'
```

Import: staqtapp_tds.studio_pyqt5.availability

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

pyqt5_availability

```
pyqt5_availability() -> 'PyQt5Availability'
```

Import: staqtapp_tds.studio_pyqt5.availability

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

require_pyqt5

```
require_pyqt5() -> 'None'
```

Import: staqtapp_tds.studio_pyqt5.availability

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

create_driver_studio_window

```
create_driver_studio_window(*, bridge: 'StudioQtBridge | None' = None, theme: 'StudioQtTheme | None' = None) -> 'DriverS
```

Import: staqtapp_tds.studio_pyqt5.main_window

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_pyqt5_shell_capability_matrix

```
studio_pyqt5_shell_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.bridge

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_cockpit_hydration_capability_matrix

```
studio_cockpit_hydration_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.hydration

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_live_event_bridge_capability_matrix

```
studio_live_event_bridge_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.live

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_live_panel_runtime_capability_matrix

```
studio_live_panel_runtime_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.runtime

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_review_workflow_capability_matrix

```
studio_review_workflow_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.review_workflow

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_evidence_timeline_capability_matrix

```
studio_evidence_timeline_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.evidence_timeline

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_risk_intelligence_capability_matrix

```
studio_risk_intelligence_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.risk_intelligence

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_manual_builder_ui_runtime_capability_matrix

```
studio_manual_builder_ui_runtime_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.manual_builder_runtime

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_export_audit_capability_matrix

```
studio_export_audit_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.export_audit

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_export_integrity_workflow_capability_matrix

```
studio_export_integrity_workflow_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.export_integrity_workflow

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_operational_stress_capability_matrix

```
studio_operational_stress_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.operational_stress

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

```
from staqtapp_tds.studio_pyqt5 import (
    StudioQtBridge,
    create_driver_studio_window,
    pyqt5_available,
)

bridge = StudioQtBridge()
shell_state = bridge.load_bundle(evidence_bundle)

if pyqt5_available():
    window = create_driver_studio_window(bridge=bridge)
    window.show()
```

18. Driver and Studio operational function index

Driver namespace

drivers.audit

audit_vm_contract

```
audit_vm_contract(package: 'BytecodePackage') -> 'None'
```

Import: staqtapp_tds.drivers.audit

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

vm_contract_table

```
vm_contract_table() -> 'Mapping[str, Mapping[str, Any]]'
```

Import: staqtapp_tds.drivers.audit

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

drivers.bytecode

compile_program

```
compile_program(program: 'TDDLProgram', *, source_material: 'str | Mapping[str, Any] | None' = None) -> 'BytecodePackage'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

compile_tddl

```
compile_tddl(source_or_program: 'str | TDDLProgram') -> 'BytecodePackage'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Validate TDDL source and compile it into a deterministic bytecode package.

decompile_to_ir

```
decompile_to_ir(package: 'BytecodePackage') -> 'TDDLProgram'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

opcode_table

```
opcode_table() -> 'Mapping[str, Mapping[str, Any]]'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

validate_bytecode_package

```
validate_bytecode_package(package: 'BytecodePackage') -> 'None'
```

Import: staqtapp_tds.drivers.bytecode

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

drivers.evidence

evidence_export_capability_matrix

```
evidence_export_capability_matrix(export_format: 'EvidenceExportFormat | str' = ) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.evidence

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.foundry

foundry_capability_matrix

```
foundry_capability_matrix(policy: 'DriverFoundryPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.foundry

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.manifest

validate_manifest

```
validate_manifest(manifest: 'DriverManifest') -> 'None'
```

Import: staqtapp_tds.drivers.manifest

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

drivers.performance

driver_vm_performance_capability_matrix

```
driver_vm_performance_capability_matrix(policy: 'DriverVMPerformancePolicy | None' = None, *, native_runner_available: 'bool')
```

Import: staqtapp_tds.drivers.performance

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

driver_vm_performance_enabled

```
driver_vm_performance_enabled(env: 'Mapping[str, str] | None' = None) -> 'bool'
```

Import: staqtapp_tds.drivers.performance

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

drivers.regression

regression_harness_capability_matrix

```
regression_harness_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.regression

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

runtime_fixture_hash

```
runtime_fixture_hash(fixtures: 'Mapping[str, Any]') -> 'str'
```

Import: staqtapp_tds.drivers.regression

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

drivers.review

batch_review_capability_matrix

```
batch_review_capability_matrix(policy: 'BatchReviewPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.review

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.runtime_manager

runtime_manager_capability_matrix

```
runtime_manager_capability_matrix(policy: 'RuntimeManagerPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.runtime_manager

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.signature

sign_payload

```
sign_payload(payload: 'bytes', *, signer: 'str', secret: 'bytes') -> 'str'
```

Import: staqtapp_tds.drivers.signature

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

verify_signature

```
verify_signature(payload: 'bytes', signature: 'str', *, signer: 'str', secret: 'bytes') -> 'bool'
```

Import: staqtapp_tds.drivers.signature

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

drivers.studio

run_studio_quick_test

```
run_studio_quick_test(source: 'str', *, signer: 'str' = 'studio-admin', secret: 'bytes' = b'local-driver-studio-secret')
```

Import: staqtapp_tds.drivers.studio

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_instruction_reference

```
studio_instruction_reference() -> 'Mapping[str, Mapping[str, object]]'
```

Import: staqtapp_tds.drivers.studio

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_readonly_capability_matrix

```
studio_readonly_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.studio

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.studio_actions

studio_admin_review_capability_matrix

```
studio_admin_review_capability_matrix(policy: 'StudioReviewSubmissionPolicy | None' = None) -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.studio_actions

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.studio_builder

studio_manual_builder_capability_matrix

```
studio_manual_builder_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.studio_builder

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

drivers.tddl

instruction_specs

```
instruction_specs() -> 'Mapping[str, InstructionSpec]'
```

Import: staqtapp_tds.drivers.tddl

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

parse_tddl

```
parse_tddl(source: 'str') -> 'TDDLProgram'
```

Import: staqtapp_tds.drivers.tddl

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

validate_tddl

```
validate_tddl(program: 'TDDLProgram') -> 'None'
```

Import: staqtapp_tds.drivers.tddl

Behavior: Validate shape, fingerprints, limits, prerequisites, and current evidence; failures are reported or raised fail-closed according to the API contract.

drivers.trace

rank_traces

```
rank_traces(traces: 'list[TraceEvidence]') -> 'list[TraceEvidence]'
```

Import: staqtapp_tds.drivers.trace

Behavior: Rank trace IDs deterministically from scores and optional confidence/depth/age inputs.

Driver Studio PyQt5 namespace

drivers.studio_builder

studio_manual_builder_capability_matrix

```
studio_manual_builder_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.drivers.studio_builder

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.availability

pyqt5_availability

```
pyqt5_availability() -> 'PyQt5Availability'
```

Import: staqtapp_tds.studio_pyqt5.availability

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

pyqt5_available

```
pyqt5_available() -> 'bool'
```

Import: staqtapp_tds.studio_pyqt5.availability

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

require_pyqt5

```
require_pyqt5() -> 'None'
```

Import: staqtapp_tds.studio_pyqt5.availability

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_pyqt5.bridge

studio_pyqt5_shell_capability_matrix

```
studio_pyqt5_shell_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.bridge

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.evidence_timeline

studio_evidence_timeline_capability_matrix

```
studio_evidence_timeline_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.evidence_timeline

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.export_audit

studio_export_audit_capability_matrix

```
studio_export_audit_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.export_audit

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.export_integrity_workflow

studio_export_integrity_workflow_capability_matrix

```
studio_export_integrity_workflow_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.export_integrity_workflow

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.hydration

manual_builder_form_schema

```
manual_builder_form_schema() -> 'tuple[StudioFormField, ...]'
```


Import: staqtapp_tds.studio_pyqt5.hydration

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_cockpit_hydration_capability_matrix

```
studio_cockpit_hydration_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.hydration

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.icons

studio_svg_icon

```
studio_svg_icon(name: 'str') -> 'str'
```

Import: staqtapp_tds.studio_pyqt5.icons

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.live

studio_live_event_bridge_capability_matrix

```
studio_live_event_bridge_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.live

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_panel_refresh_contracts

```
studio_panel_refresh_contracts() -> 'tuple[StudioPanelRefreshContract, ...]'
```

Import: staqtapp_tds.studio_pyqt5.live

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.main_window

create_driver_studio_window

```
create_driver_studio_window(*, bridge: 'StudioQtBridge | None' = None, theme: 'StudioQtTheme | None' = None) -> 'Drivers'
```

Import: staqtapp_tds.studio_pyqt5.main_window

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_pyqt5.manual_builder_runtime

studio_manual_builder_ui_runtime_capability_matrix

```
studio_manual_builder_ui_runtime_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.manual_builder_runtime

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_qt_visual_quality_capability_matrix

```
studio_qt_visual_quality_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.manual_builder_runtime

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_qt_visual_quality_review

```
studio_qt_visual_quality_review(*, form_fields: 'Sequence[StudioFormField] | None' = None, source: 'str' = '', theme: 'S'
```

Import: staqtapp_tds.studio_pyqt5.manual_builder_runtime

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.operational_stress

studio_operational_stress_capability_matrix

```
studio_operational_stress_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.operational_stress

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.panels.descriptors

panel_descriptor

```
panel_descriptor(kind: 'StudioPanelKind | str') -> 'StudioPanelDescriptor'
```

Import: staqtapp_tds.studio_pyqt5.panels.descriptors

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

studio_pyqt5.review_workflow

studio_review_rationale_templates

```
studio_review_rationale_templates() -> 'tuple[StudioReviewRationaleTemplate, ...]'
```

Import: staqtapp_tds.studio_pyqt5.review_workflow

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_review_workflow_capability_matrix

```
studio_review_workflow_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.review_workflow

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.risk_intelligence

studio_risk_intelligence_capability_matrix

```
studio_risk_intelligence_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.risk_intelligence

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

studio_pyqt5.runtime

studio_live_panel_runtime_capability_matrix

```
studio_live_panel_runtime_capability_matrix() -> 'Mapping[str, bool]'
```

Import: staqtapp_tds.studio_pyqt5.runtime

Behavior: Return a JSON-safe capability or presentation contract for Studio integration.

19. Centralized Browser and local admin control

The Browser is the centralized operational view for the TDS engine and CSV/Interpole telemetry. The server is intentionally small: HTML/CSS/JS assets are packaged, `/status.json` serves snapshots, and configuration mutations require local authority, same-origin checks, and CSRF tokens.

AdminController

```
AdminController(registry: 'ConfigRegistry | None' = None, auth: 'LocalAuthProvider | None' = None, audit: 'AuditLog | None' = None)
```

AdminController.status

```
status(self) -> 'dict[str, Any]'
```

Public operational method on this integration surface.

AdminController.stage_config

```
stage_config(self, candidate: 'RuntimeConfig', grant: 'ConfigGrant') -> 'RuntimeConfig'
```

Public operational method on this integration surface.

AdminController.promote_config

```
promote_config(self, grant: 'ConfigGrant') -> 'RuntimeConfig'
```

Public operational method on this integration surface.

AdminController.rollback_config

```
rollback_config(self, grant: 'ConfigGrant') -> 'RuntimeConfig'
```

Public operational method on this integration surface.

AdminPanelServer

```
AdminPanelServer(control: 'AdminController | None' = None, host: 'str' = '127.0.0.1', port: 'int' = 8765)
```

Local ThreadingHTTPServer wrapper. Construct with an AdminControl whose observation_source exposes snapshot methods.

AdminPanelServer.serve_forever

```
serve_forever(self)
```

Public operational method on this integration surface.

```
from staqtapp_tds.admin import AdminControl, AdminPanelServer

control = AdminControl(observation_source=fs)
server = AdminPanelServer(
    control=control,
    host="127.0.0.1",
    port=8765,
)
server.serve_forever()
```

Production binding

The default local authentication provider is suitable for localhost development. Do not expose the Browser beyond localhost without replacing development secrets and enforcing the surrounding network/security policy.

20. Configuration, native management, security, and low-level calls

RuntimeConfig

```
RuntimeConfig(config_id: 'str', generation: 'int' = 1, created_at: 'float' = , chunk_bytes: 'int' = 65536, compression:
```

Immutable runtime configuration. Stage and promote through ConfigRegistry/AdminControl rather than modifying live settings in place.

RuntimeConfig.next_generation

```
next_generation(self, **changes: 'Any') -> "'RuntimeConfig'"
```

Public operational method on this integration surface.

ConfigRegistry

```
ConfigRegistry(initial: 'RuntimeConfig | None' = None)
```

ConfigRegistry.active

```
active(self) -> 'RuntimeConfig'
```

Public operational method on this integration surface.

ConfigRegistry.stage

```
stage(self, candidate: 'RuntimeConfig') -> 'RuntimeConfig'
```

Public operational method on this integration surface.

ConfigRegistry.promote

```
promote(self, candidate: 'RuntimeConfig | None' = None) -> 'RuntimeConfig'
```

Public operational method on this integration surface.

ConfigRegistry.rollback

```
rollback(self) -> 'RuntimeConfig'
```

Public operational method on this integration surface.

ConfigRegistry.snapshot

```
snapshot(self) -> 'dict[str, Any]'
```

Return a JSON-safe snapshot.

NativeEngineManager

```
NativeEngineManager(*, expected_abi: 'int' = 1) -> 'None'
```

SecureParams

```
SecureParams(values: 'Mapping[str, str]' = ) -> None
```

Security-related parameter value object.

20.1 Advanced root-level functions

encode_header

```
encode_header(ts_create: 'int', ts_mod: 'int', flags: 'int', fmt_id: 'int', subdir_count: 'int', entry_count: 'int') ->
```

Import: staqtapp_tds.tds_filesystem

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

decode_header

```
decode_header(raw: 'bytes') -> 'dict'
```

Import: staqtapp_tds.tds_filesystem

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

load_manifest

```
load_manifest(folder: 'Path', *, inherit: 'bool' = True) -> 'ManifestPolicy'
```

Import: staqtapp_tds.manifest

Behavior: Load an existing committed artifact or report and reconstruct its typed representation.

write_default_manifest

```
write_default_manifest(folder: 'Path', *, overwrite: 'bool' = False) -> 'Path'
```

Import: staqtapp_tds.manifest

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

route_id_for

```
route_id_for(stamp: 'str', path: 'str' = '') -> 'int'
```

Import: staqtapp_tds.srz

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

classify_latency

```
classify_latency(actual_ns: 'int', expected_ns: 'int', soft_limit_ns: 'int' = 0, hard_limit_ns: 'int' = 0) -> 'LatencyBu'
```

Import: staqtapp_tds.latency

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

latency_ratio

```
latency_ratio(actual_ns: 'int', expected_ns: 'int') -> 'float'
```

Import: staqtapp_tds.latency

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

known_result_codes

```
known_result_codes() -> 'tuple[str, ...]'
```

Import: staqtapp_tds.result

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

is_known_result_code

```
is_known_result_code(code: 'TDSResultCode | str') -> 'bool'
```

Import: staqtapp_tds.result

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

result_info

```
result_info(code: 'TDSResultCode | str') -> 'TDSResultInfo | None'
```

Import: staqtapp_tds.result

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

choose_variable_kind

```
choose_variable_kind(value: 'Any') -> 'PayloadKind'
```

Import: staqtapp_tds.serializers

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

content_hash_bytes

```
content_hash_bytes(raw: 'bytes', algorithm: 'str' = 'sha256') -> 'str'
```

Import: staqtapp_tds.serializers

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

dumps_pickle

```
dumps_pickle(value: 'Any', policy: 'PicklePolicy | None' = None) -> 'bytes'
```

Import: staqtapp_tds.tds_pickle

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

loads_pickle

```
loads_pickle(raw: 'bytes', policy: 'PicklePolicy | None' = None) -> 'Any'
```

Import: staqtapp_tds.tds_pickle

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

pickle_policy_snapshot

```
pickle_policy_snapshot(policy: 'PicklePolicy | None' = None) -> 'dict[str, Any]'
```

Import: staqtapp_tds.tds_pickle

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

get_default_serialization_manager

```
get_default_serialization_manager() -> 'SerializationManager'
```

Import: staqtapp_tds.serialization.manager

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

query_requires_selector

```
query_requires_selector(**selectors) -> 'TDSResult'
```

Import: staqtapp_tds.cluster

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

create_spiral_run

```
create_spiral_run(root, run_id: 'str', *, problem: 'Any | None' = None, problem_id: 'str' = '', description: 'str' = '',
```

Import: staqtapp_tds.spiral.run

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

verify

```
verify(target: 'Any') -> 'HealthReport'
```

Import: staqtapp_tds.verify

Behavior: Run the TDS health verifier against a supported target and return a structured report.

estimate_pressure

```
estimate_pressure(counters: 'Mapping[str, int]', *, queue_depth: 'int' = 0, snapshot_lag: 'int' = 0, swiss_probe_pressur
```

Import: staqtapp_tds.asi

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

native_diagnostics_available

```
native_diagnostics_available() -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

native_diag_snapshot

```
native_diag_snapshot(*, event_limit: 'int' = 32) -> 'NativeDiagnosticSnapshot'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_reset

```
native_diag_reset() -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_set_enabled

```
native_diag_set_enabled(enabled: 'bool') -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_mark_degraded

```
native_diag_mark_degraded(degraded: 'bool' = True) -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

native_diag_emit

```
native_diag_emit(event: 'DiagnosticEvent | int', value_a: 'int' = 0, value_b: 'int' = 0) -> 'bool'
```

Import: staqtapp_tds.diagnostics

Behavior: Control or inspect the bounded native diagnostic side channel; not a storage data operation.

calculate_pressure_snapshot

```
calculate_pressure_snapshot(counters: 'Mapping[str, Any]', *, native_counters: 'Mapping[str, Any] | None' = None, perform...
```

Import: staqtapp_tds.pressure

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

build_recovery_plan

```
build_recovery_plan(pressure: 'Mapping[str, Any] | None', *, native_diagnostics: 'Mapping[str, Any] | None' = None, perform...
```

Import: staqtapp_tds.recovery

Behavior: Convert pressure/diagnostic evidence into a non-mutating recovery plan.

get_native_manager

```
get_native_manager() -> 'NativeEngineManager'
```

Import: staqtapp_tds.native.manager

Behavior: Public operational helper. Use the exact signature and returned typed object shown here; inspect the returned status/ok fields before continuing.

native_status_result

```
native_status_result() -> 'TDSResult'
```

Import: staqtapp_tds.native.manager

Behavior: Return a non-halting result describing native engine availability and state.

native_capabilities_result

```
native_capabilities_result() -> 'TDSResult'
```

Import: staqtapp_tds.native.manager

Behavior: Return a non-halting result containing the native capability matrix.

21. Integration recipes

21.1 Persist an AI checkpoint atomically

```
checkpoint = fs.makedirs("/checkpoints/current")
checkpoint.write_json("planner", planner_state, overwrite=True)
checkpoint.write_json("memory", memory_state, overwrite=True)
checkpoint.write_text("summary", summary, overwrite=True)

health = verify(fs)
if health.status != "healthy":
    raise RuntimeError(health)

TDSPersistence("./state").flush(fs, parallel_nodes=True)
```

21.2 Read many model inputs from one .tds node

```
with_reader = TDSReader("./state/agent__inputs.tds")
try:
    values = with_reader.read_many([
        "/agent/inputs/system_prompt",
        "/agent/inputs/tool_policy",
        "/agent/inputs/context_window",
    ])
finally:
    with_reader.close()
```

21.3 CSV evidence chain helper

```
from staqtapp_tds.csv_layer import (
    commit_csv_interpole_determinant_vector_report,
    commit_csv_interpole_timeline_report,
    commit_csv_interpole_timeline_ring_report,
    commit_csv_kernel_performance_gate_report,
    commit_csv_kernel_readiness_contract_report,
    commit_csv_native_row_anchor_kernel_report,
    commit_csv_native_scan_kernel_prototype_report,
    commit_csv_native_storage_artifacts,
    commit_csv_native_storage_revalidation_report,
    commit_csv_storage_adapter_replay_report,
    commit_csv_storage_bridge_manifest,
)

def prepare_semantic_ready_csv(directory, csv_id, *, chunk_size=7):
    reports = (
        commit_csv_storage_bridge_manifest(directory, csv_id),
        commit_csv_storage_adapter_replay_report(directory, csv_id),
        commit_csv_native_storage_artifacts(directory, csv_id),
        commit_csv_native_storage_revalidation_report(directory, csv_id),
        commit_csv_interpole_timeline_report(directory, csv_id),
        commit_csv_interpole_determinant_vector_report(directory, csv_id),
        commit_csv_interpole_timeline_ring_report(directory, csv_id),
        commit_csv_kernel_readiness_contract_report(
            directory, csv_id, chunk_size=chunk_size
        ),
        commit_csv_native_scan_kernel_prototype_report(
            directory, csv_id, chunk_size=chunk_size
        ),
        commit_csv_native_row_anchor_kernel_report(
            directory, csv_id, chunk_size=chunk_size
        ),
        commit_csv_kernel_performance_gate_report(directory, csv_id),
    )
    failures = [r for r in reports if not r.ok]
    if failures:
        raise RuntimeError(f"CSV evidence chain failed: {failures}")
    return reports
```


22. API selection matrix

Programming need	Use	Avoid
Store arbitrary application state	TDSDirectory.write_result / read_result	Direct mutation of internal _entries.
Store interoperable external data	write_json / write_text / raw bytes	Untrusted pickle payloads.
Persist a tree	TDSPersistence.flush	Writing .tds headers/slots manually.
Fast persisted lookup	TDSReader.read / read_many	Loading and decoding the entire file yourself.
Append model context	stalkvar("~name", value)	Inventing uncontrolled sibling names.
Rank trace candidates	rank_traces / rank_trace_run	Unstable ad-hoc sorting when tie behavior matters.
Import CSV	import_csv_bytes / import_csv_file	One TDS entry per cell.
Publish CSV artifact family	CSV transaction or commit_* calls	Partial manual writes under final keys.
Inspect native status	native_status_result / native_capabilities_result	Assuming the compiled extension exists.
Create driver proposal	DriverFoundry.propose_driver	Executing generated Python source.
Execute trusted bytecode	DriverRuntimeManager.execute_package	Giving Studio or Browser direct execution authority.
Show operational state	AdminPanelServer / StudioQtBridge	Walking storage locks from a UI refresh loop.

23. Stable boundaries to preserve

- Storage hot path performs narrow mechanical work; observers consume copied counters, events, and immutable snapshots.
- TDSResult-first APIs are the default for long-running AI services where expected failures must not halt the process.
- CSV original bytes remain preserved; canonical exports and all later evidence are derived artifacts.
- Native CSV paths are parity-gated against the Python reference path.
- Semantic IR requires explicit caller declarations and authorization. TDS validates process and lineage, not truth.
- Driver Foundry and Studio cannot sign or activate drivers. Runtime Manager, Review Board, Registry, and signature policy retain authority.
- Browser and Studio do not bypass storage, policy, or cryptographic boundaries.

Current Semantic IR scope

v3.5.2 is an in-memory review contract for candidates, lifecycle transitions, and atomic batches. Durable Semantic IR ledger persistence is not part of the current release.

24. Validation basis and compatibility

This guide was built against the Staqtapp-TDS v3.5.2 source delivery. The release validation record reports 683 passed / 11 skipped in fallback mode, 694 passed with the native extensions, 61 packaged Semantic IR tests passed, fresh-archive release checks passed, and packaged native builds passed.

The code snippets in the README and the primary storage/variable/ranking/CSV snippets in this guide were executed against the same source tree during documentation validation.

Surface	Compatibility expectation
Python fallback	Core storage, persistence, CSV reference scans, semantics, Driver systems, Browser, and headless Studio remain available.
Native index extension	Optional acceleration and native diagnostics/capability evidence.
CSV native scan extension	Optional parity-gated scan and row-anchor acceleration.
PyQt5	Optional graphical Driver Studio; headless Studio APIs remain import-safe without it.
Historical v3.5.0/v3.5.1 semantic objects	v3.5.2 validates compatible lineage and records the upgrade in newly produced values.

25. Final programmer checklist

- Use TDSDirectory result-first calls at application boundaries.
- Choose explicit text/JSON/raw formats for externally supplied data.
- Flush complete directory trees through TDSPersistence and close TDSReader objects.
- Treat validation/replay reports as gates, not decorative telemetry.
- Keep CSV source immutable and publish only complete artifact families.
- Do not force native mode unless native absence must be a failure.
- Keep semantic declarations, reviewer authorization, and evidence references explicit.
- Route driver execution through Runtime Manager and driver authority through Review Board/Registry/signature policy.
- Use the centralized Browser and Studio as observation/review surfaces, not direct storage authority.

For exhaustive passive class fields and complete class-by-class inventory, open `Staqtapp_TDS_API_Surface_Reference.pdf`.